



M.SC. DATA ANALYTICS & TECHNOLOGIES

DAT7304 - BUSINESS ANALYTICS

ASSESSMENT 1

STUDENT NAME: VICTOR ENE

STUDENT NUMBER: 2318217

MODULE TUTOR: ANCHAL GARG

SUBMITTED: 11/05/2025

WORD COUNT: 7453

List of Figures

Figure 1. CRISP - DM Model (Taylor, 2017)	7
Figure 2. Importing and structuring the fuel price dataset.....	15
Figure 3. Fuel data structure.	15
Figure 4. Missing values.....	16
Figure 5. Missing values after linear interpolation	17
Figure 6. No missing values	17
Figure 7. Boxplot showing outliers	18
Figure 8. Line plot showing outliers.....	19
Figure 9. Line plot showing no outliers.....	19
Figure 10. Outlier Detection Code.....	20
Figure 11. Duplicates.....	20
Figure 12. Duplicates code snippet	21
Figure 13. STL decomposition plot	22
Figure 14. HP Filter decomposition	23
Figure 15. Decomposition code.....	23
Figure 16. Stationarity testing with ADF.	24
Figure 17. ADF test result	25
Figure 18. ADF test result after differencing.....	25
Figure 19. ACF / PACF plots	26
Figure 20. SARIMA fitting and forecasting.....	27
Figure 21. ARIMA forecast plot	27
Figure 22. SARIMA actual vs predicted code	28
Figure 23. SARIMA actual vs predicted plot	28

Figure 24. Fitting Holt - Winters	29
Figure 25. Holt - Winters forecast	29
Figure 26. Fitting STLF.....	30
Figure 27. STLF plot	30
Figure 28. External Data	32
Figure 29. Fitting SARIMAX.....	33
Figure 30. Fitting DLM	34
Figure 31. VARSelect result	35
Figure 32. Fitting VAR	35
Figure 33. Formatting the data for SVR and RF	36
Figure 34. Fitting SVR.....	36
Figure 35. Fitting RF.....	37
Figure 36. Model evaluation.....	38
Figure 37. Model comparison bar plot	38
Figure 38. SVR: Actual vs Predicted	39
Figure 39. Forecasted SVR values	40
Figure 40. SVR historical and forecasted plot.....	41
Figure 41. Granger causality.....	42
Figure 42. Road sales influence	43
Figure 43. Road sales influence result	43
Figure 44. Reading and reprocessing data for optimisation	49
Figure 45. Extracting unique routes and transportation modes.....	49
Figure 46. Average shipping costs by route and mode.....	49
Figure 47. Current costs by Routes and Transportation Mode.....	50

Figure 48. Total current cost code snippet.....	50
Figure 49. Total current cost	50
Figure 50. Creating the cost matrix	51
Figure 51. Quantity to ship per route-mode combination	51
Figure 52. Defining capacity constraints for each transportation mode	51
Figure 53. Mode Capacity.....	52
Figure 54. Setting route demand constraints to ensure all orders are fulfilled	52
Figure 55. Solving the linear programming problem	52
Figure 56. Extracting the optimal solution.....	53
Figure 57. Solution Matrix	53
Figure 58. Cost Comparison	54
Figure 59. Mode Utilisation Analysis	54
Figure 60. Route Utilisation Analysis	55

Table of Contents

Abstract	vii
1. Introduction.....	1
1.1 Background of the Study	1
1.2 Business Problem Statement.....	2
1.3 Significance of the Problem	2
1.4 Business Questions	4
2. Literature Review	5
3. Methodology	7
3.1 CRISP DM	7
3.2 Dataset.....	9
3.3 Model Descriptions.....	10
4. Implementation and Results	14
4.1 Data Preprocessing and Exploratory Data Analysis	14
4.1.1 Data Import.....	14
4.1.2 Missing Values.....	16
4.1.3 Outlier Detection	17
4.1.4 Duplicates	20
4.1.5 Decomposition	21

4.1.6 Stationarity.....	24
4.1.7 ACF/PACF	25
4.2 Univariate Models	26
4.2.1 SARIMA	26
4.2.2 Holt-Winters.....	29
4.2.3 STLF	30
4.3 Multivariate Time Series Analysis	31
4.4 Model Evaluation.....	37
4.5 Forecast	39
4.6 Summarisation of Business Question I.....	41
4.7 Summarisation of Business Question II.....	42
4.8 Summarisation of Business Question III.....	42
4.9 Summarisation of Business Question IV	43
4.10 Summarisation Business of Question V	44
5. Conclusion and Recommendations	45
6. Part B: Solution to Optimisation problem	47
6.1 Problem Statement.....	47
6.2 Optimisation Problem.....	47
6.2.1 Objective Function	47

6.2.2 Constraints	48
6.2.3 Decision Variables	48
6.2.4 Parameters	48
6.3 Implementation	49
6.3.1 Data Preparation	49
6.3.2 Model Setup.....	51
6.4 Result and Analysis	52
6.4.1 Solution Matrix.....	52
6.4.2 Cost Comparison	53
6.4.3 Mode Utilisation Analysis.....	54
6.4.4 Route Utilisation Analysis.....	55
6.4.5 Key Insights	55
6.4.6 Constraint Satisfaction	56
6.5 Recommendations.....	56
6.6 Conclusion	56
7. Personal Reflection.....	58
References.....	59

Abstract

This report addresses two distinct business challenges: forecasting fuel prices and optimising transportation costs within a multi-supplier, multi-warehouse supply chain. Part A utilises time series analysis on weekly UK fuel price data (January 2021 - February 2025), integrating Brent crude oil prices as an external variable. Through the CRISP-DM framework, various univariate (SARIMA, STLF, Holt-Winters) and multivariate (VAR, DLM, SARIMAX, SVR, Random Forest) models were modelled and evaluated. Support Vector Regression (SVR) demonstrated the highest accuracy, predicting a moderate decline in fuel prices over a 24-week forecast horizon. The analysis also confirmed the significant influence of crude oil prices on UK fuel markets and the beneficial impact of including road fuel sales in multivariate forecasts. Part B tackles the problem of minimising transportation costs using linear programming in R. Formulating the objective as minimising total shipping costs across routes and transportation modes (Road, Air, Rail, Sea), subject to demand fulfillment and mode capacity constraints, the optimisation model achieved a 9.49% cost reduction compared to the initial way the transportation was setup.

1. Introduction

1.1 Background of the Study

Fuel prices frequently rise and fall, with these fluctuations influencing economic systems at both the global and national levels. Globally, changes in fuel prices impact international trade dynamics and crude oil markets, while nationally, they affect domestic transport, inflation rates, and consumer spending behaviour (Hamilton, 2010; Kilian, 2008). These variations can significantly influence individual and business expenses while also serving as a critical indicator that shape public fiscal strategies and organisational decision-making frameworks (Sterner, 2012).

Within the United Kingdom, fuel pricing holds heightened relevance due to the impact it has on transportation logistics, its direct correlation with inflation and its role as a key indicator of broader energy market trends (Baumeister and Peersman, 2013). This analysis uses weekly UK fuel price data from January 2021 to February 2025 to identify trends and forecast future values. The dataset includes average fuel prices, road fuel sales and estimated quantity demand per transaction. An external dataset of Brent crude oil prices is also merged to the data for further time series analysis.

Time series analysis has emerged as a powerful technique in business analytics, enabling analysts to make data-driven forecasts based on historical trends and patterns (Hyndman and Athanasopoulos, 2021). With the availability of high frequency fuel price data in the UK and access to exogenous variables such as global crude oil prices, it is now possible to construct robust models that can provide short and long term forecasts with high accuracy. This report applies various techniques relating to time series to analyse, uncover patterns, model and forecast weekly average fuel prices in the UK, aiding stakeholders in strategic decision-making.

1.2 Business Problem Statement

Fluctuating fuel prices present significant challenges for UK fuel retailers, policymakers and consumers. The business in focus is a UK fuel retail chain operating a network of stations with key stakeholders including retail managers, Logistics providers, policy makers and consumers. With the availability of historical fuel price data, there is need for a robust forecasting system to help stakeholders anticipate future price movements and make informed policy or pricing decisions.

Can we accurately forecast UK average fuel prices using historical time-series data and relevant external variables such as crude oil prices? This question is of strategic importance for stakeholders, as an effective forecasting model can enable better budgetary planning, inventory management and policy formation. Businesses can develop contingency plans for fuel shortages and consumers gain access to transparent pricing trends to inform budgeting decisions.

1.3 Significance of the Problem

Fuel prices pose systemic risks to economic stability, supply chain efficiency and household financial planning, fuel costs directly influence inflation, transport logistics and energy policy (Hamilton, 2010), the inability to anticipate price movements can create vulnerabilities across sectors. The significance of this problem becomes evident through its impact on:

- I. Government Budget Planning: Fuel price volatility makes fiscal strategies more difficult since it casts doubt on the distribution of subsidies and the execution of price stabilisation plans. When it comes to determining macroeconomic policy responses, accurate forecasting is essential (Baumeister and Kilian, 2016).

- II. Private Sector Logistics: Fuel retailers and transport firms often encounter operational challenges, including inventory imbalances and unexpected fuel surcharge costs.
- III. Consumer Welfare: Unpredictable fuel prices make it harder for households, particularly those on low incomes, to balance their budgets.
- IV. Inflation Control: Fuel prices are a key driver of inflation, influencing transport, manufacturing, and commodity costs. Unchecked volatility risks broader economic instability (Hamilton, 2010).

This report employs advanced time-series methodologies to address stakeholder needs by predicting average fuel prices 24 weeks into the future, enabling fuel retailers to optimise pricing strategies. This reduces financial losses in sudden price drops, while logistics firms can align transport schedules with anticipated fuel costs to minimise operational disruptions and improve cost analysis.

Retailers can leverage the revealed seasonal patterns and trends via decomposition techniques to time promotional campaigns or adjust stockpiling strategies and enhance competitiveness. Additionally, the analysis quantifies the causal impact of crude oil prices on UK fuel markets using Granger causality tests and multivariate models (Kilian and Zhou, 2020). Together, these outputs equip stakeholders with tools to mitigate volatility-driven risks and support long term economic resilience.

1.4 Business Questions

The following business questions guide the analysis and modelling in this report:

- I. Are there trends and seasonal patterns in UK average fuel prices?
- II. Do crude oil prices influence average fuel prices in the UK?
- III. Does average road fuel sales volume influence the accuracy of multivariate time-series forecasts?
- IV. Which forecasting model provides the best accuracy for UK fuel prices?
- V. What are the forecasted average fuel prices for 12 weeks into the future using historical patterns?

2. Literature Review

Accurate fuel price forecasting is critical for policymakers, businesses and consumers, particularly in oil-importing economies like the UK, where fuel prices influence inflation, transportation costs and economic stability (Hamilton, 2010). Crude oil is at the heart of these price dynamics and plays a central role in driving price fluctuations with Brent crude serving as a critical global benchmark, especially for European markets. As a major driver of refined petroleum product pricing, Brent crude prices directly shape cost structure across the supply chain and as such, understanding and predicting its impact is fundamental to any fuel price forecasting strategy (Baumeister and Kilian, 2015). Recent geopolitical shocks (Russia-Ukraine crisis, COVID-19) and energy transitions necessitate the need for robust models to address structural breaks and uncertainty (Moyo, Anyikwa and Phiri, 2023).

Time-series analysis has long been integral to business forecasting and is widely used to model fuel price dynamics. The foundational work of Box and Jenkins (1976) established Autoregressive Integrated Moving Average (ARIMA) models, which laid foundation for researchers to make advancements in seasonal (SARIMA), exogenous-inclusive (ARIMAX) and multivariate (VAR) extensions. Methodologies have evolved from traditional models such as ARIMA and VAR to machine learning and hybrid approaches, which better capture non-linearities and volatility (Hyndman and Athanasopoulos, 2021). In fuel pricing research, both univariate and multivariate methods have been explored. For instance, Lin and Li (2015) identified predictive links between global oil prices and domestic fuel costs using Granger causality tests. Similarly, Chao et al. (2020) demonstrated that multivariate frameworks integrating crude oil prices and macroeconomic

variables such as exchange rates and crude oil price yield superior accuracy to conventional ARIMA models.

Machine learning techniques, such as Support Vector Regression (SVR), Random Forests, ANNs and LSTMs, have also gained traction for capturing non-linear relationships in fuel price data (Gupta and Nigam, 2020). Guo et al. (2019) argue that these non-linear machine learning models often outperform statistical models when dealing with complex, non-linear relationships in the data, although their interpretability remains a challenge. The research by Jha *et al.* (2024) investigated crude oil price forecasting using multivariate techniques, with a focus on overcoming the limitations of traditional econometric models. The study found that Support Vector Regression (SVR) with a radial basis function (RBF) kernel outperformed other methods, demonstrating superior stability and accuracy in capturing the non-linear and time-varying dynamics of crude oil prices.

This report builds on existing literature by comparing a comprehensive suite of forecasting models, from classical statistical to modern machine learning approaches, while also considering the influence of exogenous variables like Brent crude oil prices and average road fuel sales.

3. Methodology

3.1 CRISP DM

The Cross-Industry Standard Process for Data Mining (CRISP-DM) is a globally accepted framework that offers a systematic approach to manage the data mining process and ensure effective execution (Putler and Krider, 2012a). Comprising of six interconnected phases, this methodology is essential for structuring data analysis tasks, such as time series prediction, by addressing the requirements of each phase. The following discussion outlines these CRISP-DM phases and their relevance to predicting average fuel prices in the United Kingdom through historical trends and external influencing factors.

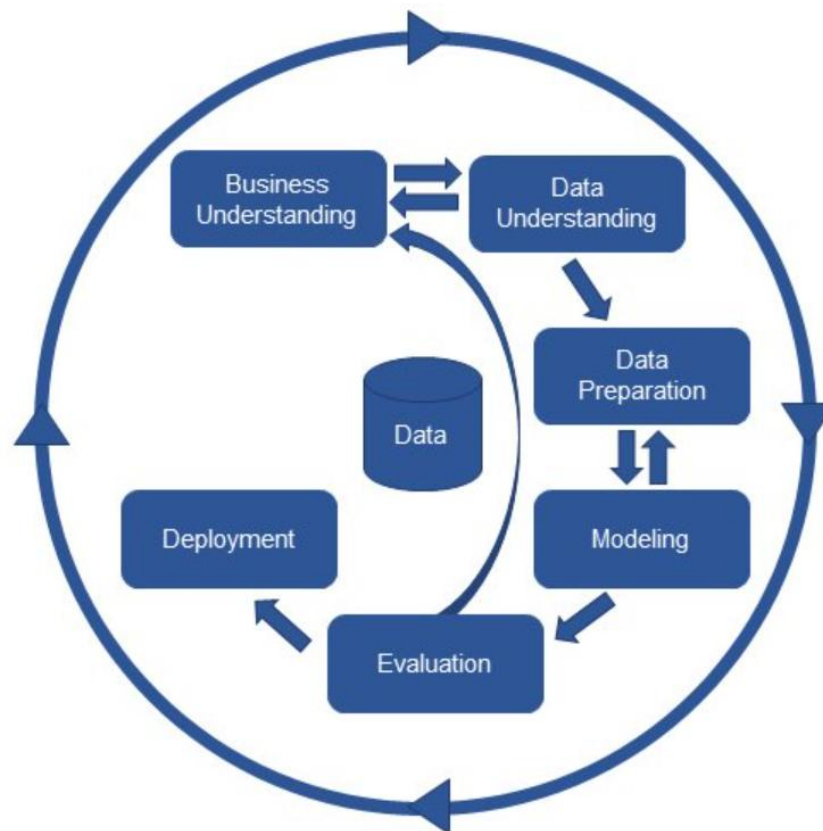


Figure 1 CRISP - DM Model (Taylor, 2017)

a. Business Understanding

The Business Understanding phase establishes the project goals aligned with stakeholder priorities and organisational needs, evaluation of the business is necessary to identify existing resources and gaps, ensuring it aligns with project requirements. This phase is critical for framing the problem and defining measurable outcomes (Schröer, Kruse and Gómez, 2021). This report addresses volatile UK fuel prices by developing a 24-week forecasting model using historical data and external variables and a structured plan with success criteria being predictive accuracy.

b. Data Understanding

The Data Understanding phase involves collecting, describing, and exploring the available data to identify data quality issues and initial insights (Schröer, Kruse and Gómez, 2021). For this analysis data has been sourced from weekly fuel prices (Jan 2021 – Feb 2025) and aggregated Brent crude oil prices supported by statistical descriptions and attribute correlations. Exploratory analysis using line plots, STL decomposition, and stationarity tests identified seasonal patterns and non-stationary trends in the data.

c. Data Preparation

The Data preparation phase involves is cleaning, transforming, defining inclusion and exclusion criteria and structuring data for modelling. The methodologies applied across these steps are model-dependent, as the choice of techniques must align with the specific requirements and assumptions of the selected algorithm (Schröer, Kruse and Gómez, 2021).

d. Modelling

The modelling phase entails selecting and applying data mining techniques tailored to the business problem and data characteristics, guided by frameworks like CRISP-DM, followed by evaluation against predefined metrics to identify the most effective model (Han, Pei and Tong, 2023).

e. Evaluation

The Evaluation phase assesses whether model outcomes align with predefined business goals, requiring interpretation of results and formulation of actionable recommendations. The entire analytical process is reviewed to identify improvements, ensuring alignment with operational workflows and stakeholder expectations (Putler and Krider, 2012). The best model is chosen based on its ability to predict future values accurately.

f. Deployment

The final work could be a report or a software, the Deployment phase involves using the chosen model for actual decision-making. This could involve integrating the model into the business process for real-time predictions or using it for policy recommendations (Schröer, Kruse and Gómez, 2021). The best-performing model, identified through evaluation was used to generate 24 week forecasts for future fuel prices.

3.2 Dataset

The primary dataset used in this study comprises weekly observations of average fuel prices, estimated quantity demand per transaction and average road fuel sales, starting from January 2021 to February 2025. An additional external dataset, containing daily Brent crude oil prices was sourced from the U.S. Energy Information Administration (EIA). This data was aggregated to a weekly averages to align with the main dataset. Both datasets are non-seasonally adjusted,

requiring preprocessing to handle missing values and outliers, the datasets were cleaned, merged and analysed using R programming.

3.3 Model Descriptions

The following models were employed to analyse and forecast UK fuel prices from January 2021 to February 2025, leveraging both traditional time-series techniques and machine learning approaches. Each model is described below.

a. SARIMA

The Seasonal Autoregressive Integrated Moving Average (SARIMA) model is an extension of the ARIMA framework, designed to capture autoregressive, differencing, moving average components and also seasonal patterns in data with periodic trends making it particularly suitable for time series data with recurring patterns (Hyndman and Athanasopoulos, 2021). such as the 52-week cyclical patterns identified in weekly UK fuel prices. The model combines non-seasonal parameters (p, d, q) and seasonal parameters (P, D, Q, s) , where p and P represent autoregressive terms, d and D represent differencing orders, q and Q represent moving average elements, and s is the seasonal interval (Hyndman and Athanasopoulos, 2021). Applied to fuel price forecasting, SARIMA captured historical trends and seasonal variations, generating reliable predictions, however, its efficacy depends on linear assumptions.

b. STLF (Seasonal Time Series Forecasting using Loess)

The STLF model is a seasonal forecasting method that applies Seasonal and Trend decomposition using Loess (STL) to decompose the time series into its seasonal, trend, and remainder components. After decomposition, exponential smoothing is applied to the seasonal and trend

components to generate forecasts. Unlike traditional ETS (Error-Trend-Seasonal) models, which directly smooth the entire time series, STLF first performs seasonal decomposition using Loess, which allows for more flexibility in handling seasonality that may change over time. This model is highly effective for weekly or daily time series that exhibit irregular seasonality patterns, as seen in many economic indicators like fuel prices (Hyndman and Athanasopoulos, 2021).

c. Holt-Winters

The Holt-Winters model (triple exponential smoothing) builds on exponential smoothing by incorporating level, trend, and seasonality adjustments through three optimised smoothing parameters to reduce forecasting errors (Chatfield, 2000). Applied to UK fuel price data, it is suitable for datasets with steady periodic patterns. However, its reliance on historical trends limits adaptability to abrupt external shocks, such as oil price volatility, which may introduce non-linear dynamics.

d. VAR

The Vector Autoregression (VAR) model is a multivariate time-series approach that models the interdependencies among multiple related variables. Unlike univariate models, VAR treats each variable (e.g., fuel prices, oil prices, road fuel sales) as a function of its own lagged values and the lagged values of all other variables in the system. In this study, VAR was employed to capture the dynamic relationships between fuel prices, Brent crude oil prices, and road fuel sales. VAR is well suited for exploring Granger causality and forecasting in interconnected systems, though it requires stationarity and can be sensitive to model specification (Lütkepohl, 2007).

e. DLM

The Dynamic Linear Model (DLM) is a state-space methodology that employs Kalman filtering to represent time-series data through a combination of observable measurements and unobserved components, such as trends, cyclical or seasonal patterns, making them highly adaptable to changing conditions permitting parameters to evolve over time and the integration of external variables (Durbin and Koopman, 2012). In this analysis, DLM was applied to model fuel prices as a state-space system, estimating latent trends and seasonal components while adapting to structural changes in the data. Using Kalman filtering enables robust handling of missing values and outliers, However DLM can be computationally intensive and require careful specification (Harvey, 2003).

f. SARIMAX

The Seasonal Autoregressive Integrated Moving Average with Exogenous Variables (SARIMAX) model is an extension of the SARIMA framework that integrates external predictors, or exogenous variables, to improve the modelling of time-series data influenced by external factors (Hyndman and Athanasopoulos, 2021). In this report, SARIMAX was used to model fuel prices while including Brent crude oil prices and road fuel sales as exogenous variables. Like SARIMA, it accounts for autoregressive, differencing, and moving average components, as well as seasonal patterns, but its ability to integrate covariates makes it more robust for multivariate analysis and computational demands also escalate with the inclusion of multiple covariates (Alharbi and Csala, 2022).

g. SVR

Support Vector Regression (SVR) is a machine learning technique that applies the principles of support vector machines to regression problems. SVR seeks to find a function that predicts continuous values rather than categorical labels while maintaining a margin of tolerance around the true data points, using a kernel most notably Radial Basis Function (RBF) to capture non-linear relationships. In this study, SVR was used to model fuel prices, incorporating lagged fuel prices, oil prices and road fuel sales as features. SVR is particularly effective for datasets with non-linear dynamics but requires careful tuning of hyperparameters like kernel selection and large datasets may escalate training times (Gupta and Nigam, 2020; Smola and Schoelkopf, 2004).

h. Random Forest

Random Forest is a machine learning model that constructs multiple decision trees during training and aggregates their predictions by majority voting or averaging to improve accuracy and robustness. In time-series forecasting, it adapts to temporal data by treating lagged values of the target variable and exogenous predictors as input features. In this report, Random Forest was applied to forecast fuel prices, leveraging features such as lagged fuel prices, oil prices, and road fuel sales. Its strength lies in its ability to capture complex, non-linear relationships and handle high-dimensional data while mitigating overfitting through randomness in tree construction (Breiman, 2001; James *et al.*, 2013).

4. Implementation and Results

4.1 Data Preprocessing and Exploratory Data Analysis

Data preprocessing is a foundational stage in the data analytics process. It ensures that datasets are properly structured, cleaned, and transformed before applying statistical analysis or predictive modelling. Without adequate preprocessing, even the most sophisticated models may produce biased, inaccurate, or unreliable results (Han, Pei and Tong, 2023). Data preprocessing involves tasks such as imputing missing values, type conversion, duplicate removal and feature transformation all of which are necessary to enhance model quality.

Exploratory Data Analysis is an essential phase in any data analysis project, it involves examining datasets using statistical techniques and visualisations to understand the main characteristics, general patterns, relationships and potential issues or anomalies before applying formal modelling or testing procedures (James *et al.*, 2013). The following steps were performed using R programming.

4.1.1 Data Import

The dataset was imported using the `read_excel` function, with preliminary metadata rows excluded. Columns were renamed to make the column names easier to work with and standardised for further analysis. The data is then converted to appropriate types using `mutate` function from `dplyr` package, `mutate` is commonly used to transform the data, creating new columns or modifying existing ones, this step ensured that numerical and categorical variables were correctly interpreted by modelling functions (Wickham, Cetinkaya-Rundel and Grolemund, 2023).

The `str()` function provides an overview of the structure of the fuel data frame, displaying the data types of all columns, the first few rows of observations and the number of observations and variables in the dataset as shown in figure 3 below, confirming that the columns have been correctly transformed and that the data is ready for analysis.

```
{r}
# Read the data, skipping the first 5 rows
fuel0 <- read_excel(
  path      = "fuel_price.xlsx",
  sheet     = "Table 1 weekly",
  skip      = 5,
  col_names = TRUE
)

# Convert types ensure variables are in the correct type
fuel <- fuel0 |>
  rename(
    week_ending      = "week ending",
    avg_fuel_price    = "Average fuel price",
    quantity_per_transaction = "Estimated quantity demand per transaction",
    avg_rd_sales      = "Average road fuel sales"
  ) |>
  mutate(
    week_ending      = as.Date(week_ending),
    avg_fuel_price    = as.numeric(avg_fuel_price),
    quantity_per_transaction = as.numeric(quantity_per_transaction),
    avg_rd_sales      = as.numeric(avg_rd_sales)
  )

str(fuel)
```

Figure 2. Importing and structuring the fuel price dataset.

```
tibble [214 × 4] (S3: tbl_df/tbl/data.frame)
 $ week_ending      : Date[1:214], format: "2021-01-31" "2021-02-07" "2021-
02-14" ...
 $ avg_fuel_price    : num [1:214] 92.6 94.5 95.9 97.4 98.3 ...
 $ quantity_per_transaction: num [1:214] 97.2 95.4 95.9 92.9 92.1 ...
 $ avg_rd_sales      : num [1:214] 65.8 67.5 67.3 69 69.6 ...
```

Figure 3. Fuel data structure.

4.1.2 Missing Values

Handling missing data is a crucial step in time-series analysis, as unaddressed gaps can distort model estimates, result in biased statistical summaries and reduce forecasting accuracy (Little and Rubin, 2020). In this report, missing values were detected in the three variables: `avg_fuel_price`, `quantity_per_transaction` and `avg_rd_sales`. These were identified using the line of code `fuel |> summarise_all(~ sum(is.na(.)))`, which summarises the number of missing entries across all columns. As shown in Figure 4, the dataset initially contained 15 missing values across the numerical features.

<code>week_endi...</code> <int>	<code>avg_fuel_price</code> <int>	<code>quantity_per_transaction</code> <int>	<code>avg_rd_sales</code> <int>
0	2	2	11

Figure 4. Missing values

To address these gaps, First linear interpolation was used to fill in missing values by estimating them based on the linearly interpolated values of neighbouring observations. This method is particularly effective for time-series data where the assumption of gradual, continuous change between time points is valid (Moritz and Bartz-Beielstein, 2017).

However, linear interpolation did not fully resolve all missing entries and in the `avg_rd_sales` column we still had 9 values missing as seen in Figure 5. Therefore, the remaining gaps were imputed using the forward-fill method (Last Observation Carried Forward or LOCF). This technique populates missing entries with the last observed non-missing value and is appropriate in cases where the most recent observation is assumed to remain valid. After applying both techniques, the dataset was fully imputed with no remaining missing values, as verified in Figure 6.

week_endi... <int>	avg_fuel_price <int>	quantity_per_transaction <int>	avg_rd_sales <int>
0	0	0	9

Figure 5. Missing values after linear interpolation

```
{r}
# Check for missing values
fuel |> summarise_all(~ sum(is.na(.)))

# Impute missing values via linear interpolation
fuel <- fuel |>
  arrange(week_ending) |>
  mutate(
    # Linear interpolation for interior NAs
    avg_fuel_price = na.approx(avg_fuel_price, na.rm = FALSE),
    quantity_per_transaction = na.approx(quantity_per_transaction, na.rm = FALSE),
    avg_rd_sales = na.approx(avg_rd_sales, na.rm = FALSE)
  )

# Impute missing values via forward fill
fuel <- fuel |>
  arrange(week_ending) |>
  mutate(
    avg_rd_sales = na.locf(avg_rd_sales, na.rm = FALSE)
  )

fuel |> summarise_all(~ sum(is.na(.)))
```

A tibble: 1 x 4

week_endi... <int>	avg_fuel_price <int>	quantity_per_transaction <int>	avg_rd_sales <int>
0	0	0	0

Figure 6. No missing values

4.1.3 Outlier Detection

Outliers are unusually high or low values, if not addressed they can distort model predictions and reduce the reliability of the results. In this analysis, we used the Interquartile Range (IQR) method to identify outliers in the avg_fuel_price column. Using this approach, we detected two clear

outliers at index positions 190 and 202, as illustrated in a boxplot in Figure 7 and a line plot in Figure 8. These values were unusually far from the rest of the data and risked skewing the analysis, so we temporarily marked them as NA (missing).

To fill in these gaps and preserve the overall continuity of the series, we applied linear interpolation. With the outliers successfully handled, the dataset was ready for the next phase of analysis.

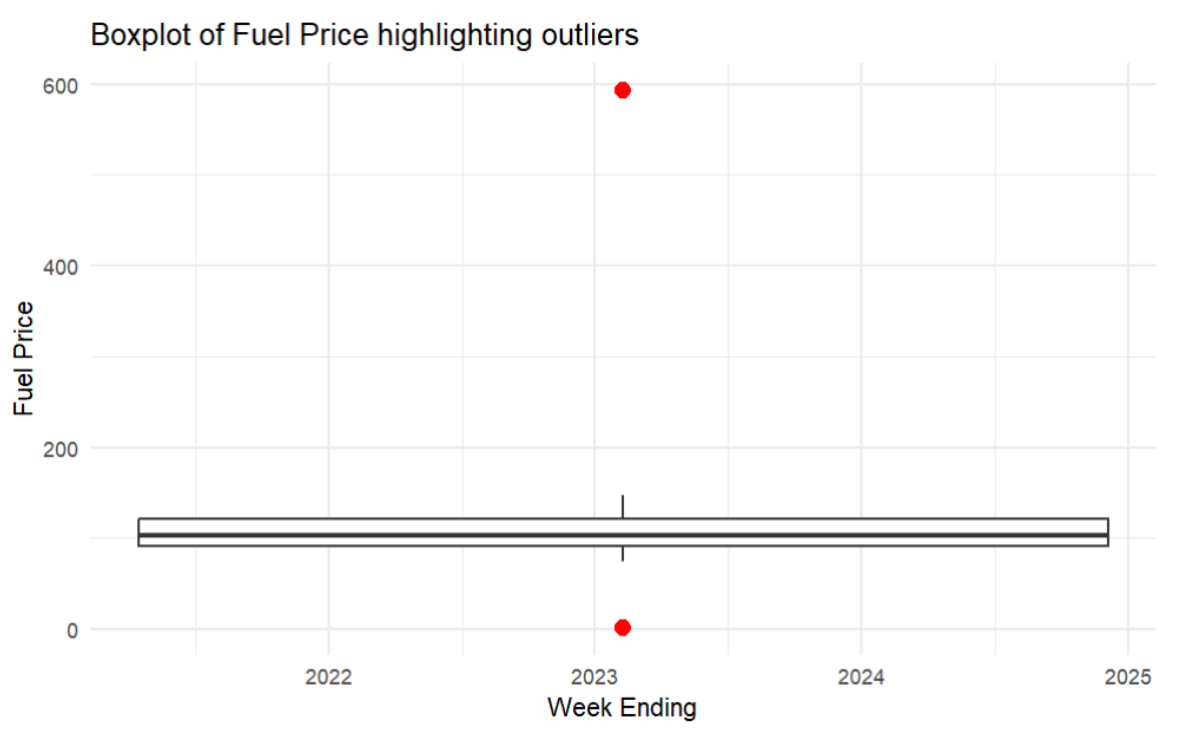


Figure 7. Boxplot showing outliers

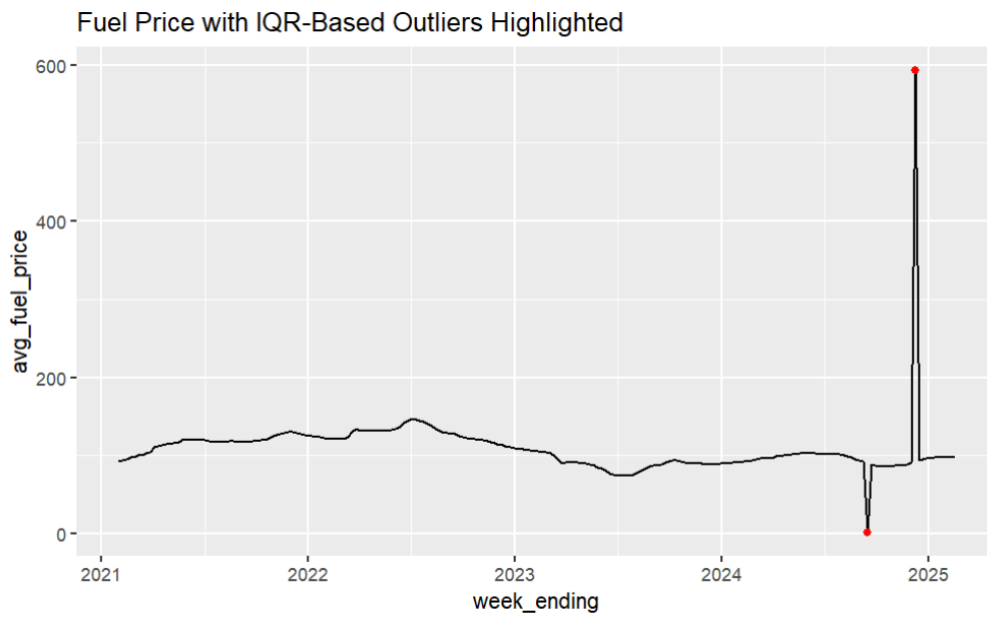


Figure 8. Line plot showing outliers

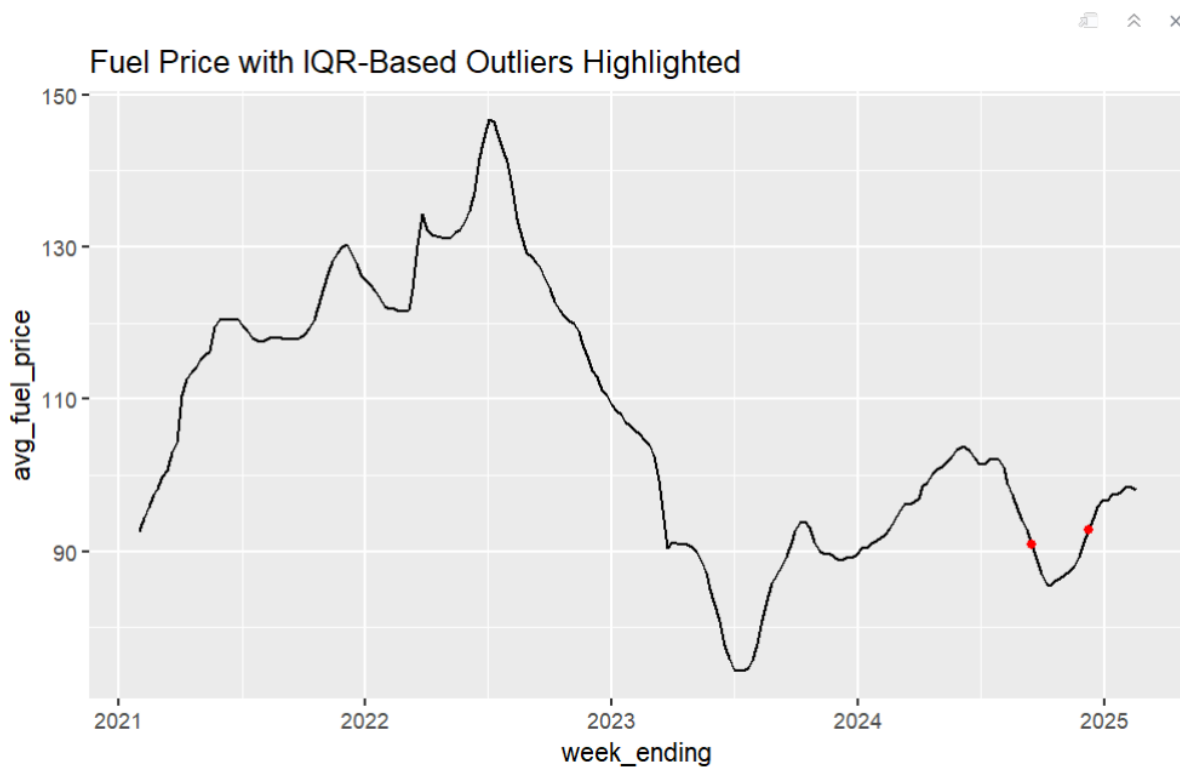


Figure 9. Line plot showing no outliers

```

# Visualize outliers using a boxplot
ggplot(fuel, aes(x = week_ending, y = avg_fuel_price)) +
  geom_boxplot(outlier.colour = "red", outlier.size = 3) +
  labs(title = "Boxplot of Fuel Price highlighting outliers", x = "Week Ending", y
= "Fuel Price") +
  theme_minimal()

# Calculate IQR values
iqr_vals <- IQR(fuel$avg_fuel_price, na.rm = TRUE)
q1      <- quantile(fuel$avg_fuel_price, 0.25, na.rm = TRUE)
q3      <- quantile(fuel$avg_fuel_price, 0.75, na.rm = TRUE)
lower_f <- q1 - 1.5 * iqr_vals
upper_f <- q3 + 1.5 * iqr_vals

# Identify outlier indices
out_idx <- which(fuel$avg_fuel_price < lower_f | fuel$avg_fuel_price > upper_f)

# Visualise IQR-based outliers on the interpolated series
ggplot(fuel, aes(week_ending, avg_fuel_price)) +
  geom_line() +
  geom_point(data = fuel[out_idx, ], aes(week_ending, avg_fuel_price), colour =
"red") +
  labs(title = "Fuel Price with IQR-Based Outliers Highlighted")

# replace the outliers with NA and apply linear interpolation
fuel$avg_fuel_price[out_idx] <- NA
fuel$avg_fuel_price <- zoo::na.approx(fuel$avg_fuel_price, na.rm = FALSE)

```

Figure 10. Outlier Detection Code

4.1.4 Duplicates

Duplicate entries were identified in the dataset. These duplicates represent multiple observations recorded for the same week which could distort the analysis. The identified duplicates are as shown in Figure 11 below.

week_ending <date>	avg_fuel_price <dbl>	quantity_per_transaction <dbl>	avg_rd_sales <dbl>
2025-02-09	98.41097	100.6244	109.094
2025-02-09	98.41097	100.6244	109.094
2025-02-16	98.03392	101.9547	109.094
2025-02-16	98.03392	101.9547	109.094

Figure 11. Duplicates

Both of these weeks had duplicate rows with identical values across all columns. To address this issue, the dataset was aggregated by week to remove the duplicates. The mean of the duplicated rows was calculated for each week, ensuring that there was only one representing entry per week based on the average of the duplicate values for each column.

```
{r}
# Find duplicates
duplicates <- fuel |>
  group_by(week_ending) |>
  filter(n() > 1)

print(duplicates)

fuel <- fuel |>
  group_by(week_ending) |>
  summarise(
    avg_fuel_price      = mean(avg_fuel_price, na.rm = TRUE),
    avg_rd_sales        = mean(avg_rd_sales, na.rm = TRUE)
  ) |>
  ungroup()

# Check again for duplicates after aggregation
duplicates_after <- fuel |>
  group_by(week_ending) |>
  filter(n() > 1)

print(duplicates_after)
```

Figure 12 Duplicates code snippet

4.1.5 Decomposition

To better understand the structure and behaviour of the fuel price time series, we performed time series decomposition. This technique breaks the data into three components: trend, seasonality, and residual, this is useful for capturing seasonal patterns and understanding the long-term trend in the data (Hyndman and Athanasopoulos, 2021). Two methods were used in this analysis, STL decomposition and the Hodrick–Prescott filter.

STL (Seasonal-Trend decomposition using Loess) is particularly useful when the data exhibits strong seasonal patterns. It separates the long-term trend, recurring seasonal cycles and any irregular noise by applying locally weighted regression (Rojo *et al.*, 2017).

In addition, we used the Hodrick–Prescott filter, a popular technique in economics for extracting smooth long-term trends from time series data. This method distinguishes between permanent trends and short-term cyclical movements. While both methods offer valuable insights, STL was especially helpful in revealing repeating seasonality in our dataset.

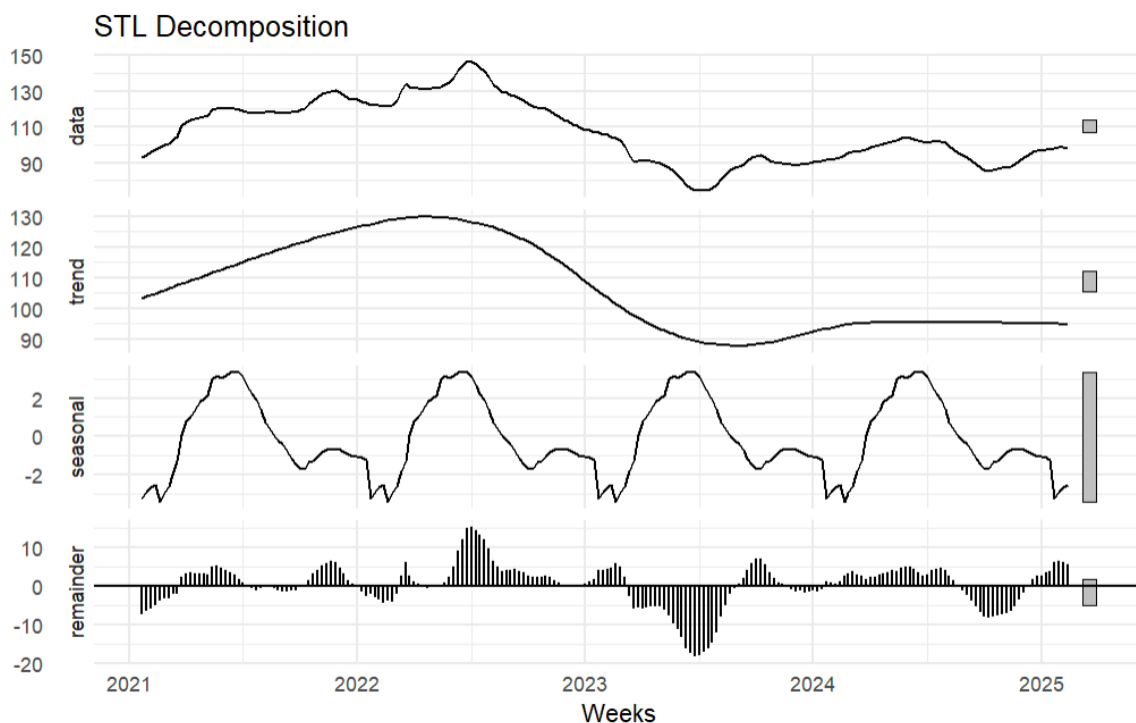


Figure 13. STL decomposition plot

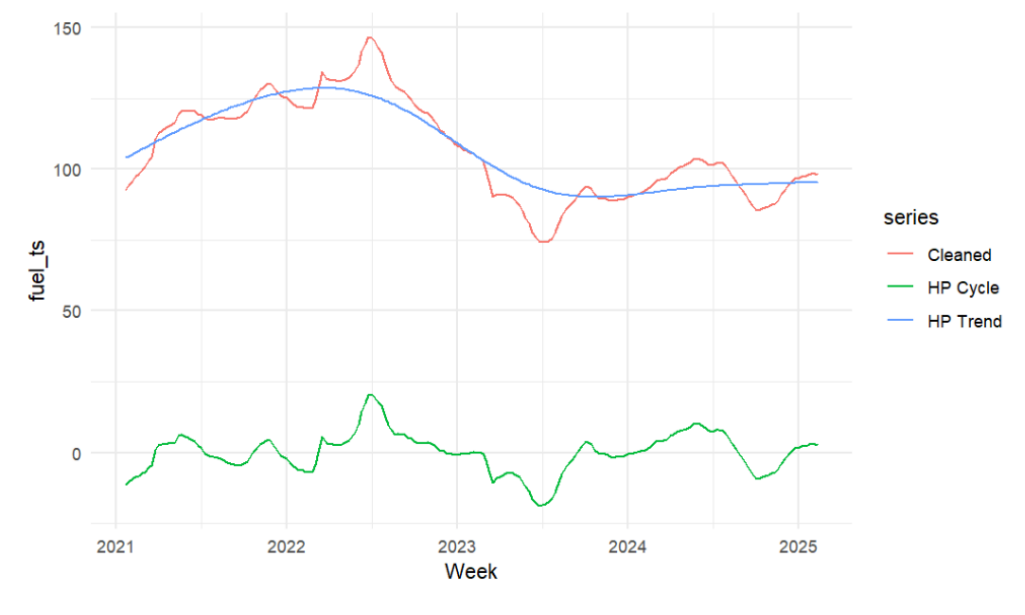


Figure 14 HP Filter decomposition

```
{r}
# STL decomposition
d_compose <- stl(fuel_ts, s.window = "periodic")
autoplot(d_compose) +
  labs(
    title = "STL Decomposition",
    x = "Weeks"
  ) +
  theme_minimal()

# Hodrick-Prescott filter decomposition
hp <- hpfilter(fuel_ts, type = "lambda", freq = 129600)
autoplot(fuel_ts, series = "Cleaned") +
  autolayer(hp$trend, series = "HP Trend") +
  autolayer(hp$cycle, series = "HP Cycle") +
  labs(
    title = "HP Filter Decomposition",
    x = "Week"
  ) +
  theme_minimal()
```

Figure 15. Decomposition code

4.1.6 Stationarity

The stationarity on the original time series `fuel_ts` was checked using the Augmented Dickey-Fuller (ADF) test, the ADF test is a statistical test used to check the stationarity of a time series, a time series is stationary if its statistical properties like mean and variance do not change over time. Many forecasting models, including ARIMA and SARIMA assume that the time series is stationary in order to perform accurately (Hyndman and Athanasopoulos, 2021).

The result showed a p-value of 0.3854, which is greater than the standard threshold of 0.05. This meant that we could not reject the null hypothesis, confirming that the series was non-stationary. To address this, we applied first-order differencing, a method that subtracts each observation from the previous one to remove trend components and stabilise the series. After differencing, we re-applied the ADF test. This time, the p-value dropped to 0.01, allowing us to reject the null hypothesis and conclude that the differenced series is stationary.

```
{r}  
adf.test(fuel_ts)  
  
fuel_ts_diff <- diff(fuel_ts)  
adf.test(fuel_ts_diff)
```

Figure 16. Stationarity testing with ADF.

Augmented Dickey-Fuller Test

```
data: fuel_ts  
Dickey-Fuller = -2.4545, Lag order = 5, p-value = 0.3854  
alternative hypothesis: stationary
```

Figure 17 ADF test result

```
data: fuel_ts_diff  
Dickey-Fuller = -4.2916, Lag order = 5, p-value = 0.01
```

Figure 18 ADF test result after differencing

4.1.7 ACF/PACF

The ACF (Autocorrelation Function) and PACF (Partial Autocorrelation Function) plots are essential for identifying suitable parameters for time series models such as ARIMA and SARIMA (Box *et al.*, 2016).

The ACF plot shows how each observation in the series is correlated with its previous values (lags). This helps us detect repeating patterns or dependencies in the data over time. If the autocorrelation remains high across several lags, it may suggest seasonality or long memory effects in the data. On the other hand, the PACF plot isolates the effect of each lag by removing the influence of intermediate lags, helping to identify the appropriate number of autoregressive (AR) terms for the model (Hyndman and Athanasopoulos, 2021).

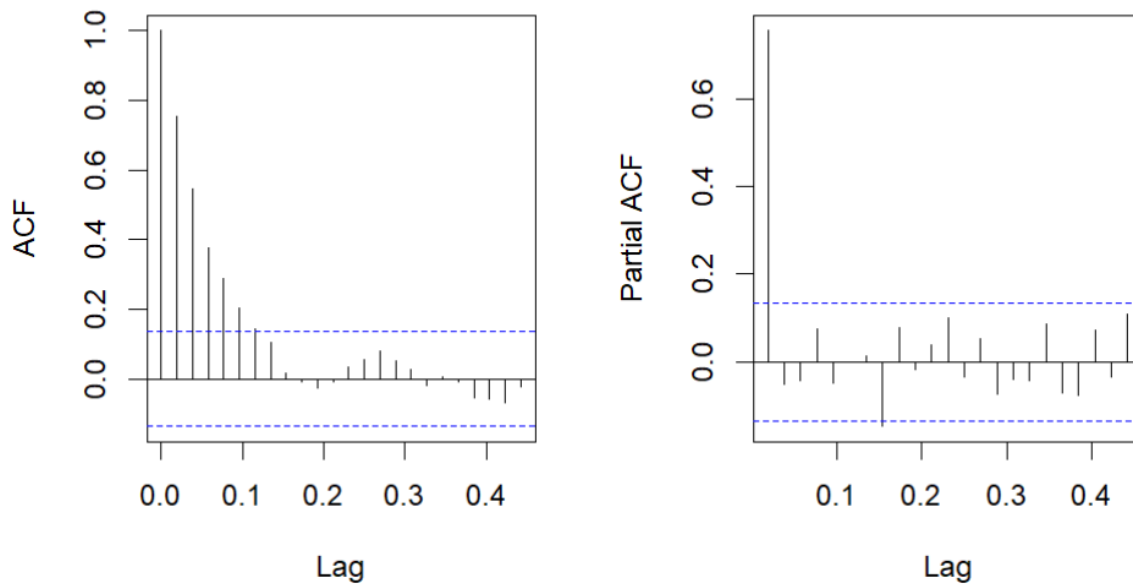


Figure 19 ACF / PACF plots

4.2 Univariate Models

Each model was trained using an 80/20 train and test split, by using 80% of the data for training, the model has enough historical data to learn patterns, trends and relationships. This allows the model to capture the underlying structure of the time series and perform well on unseen data.

4.2.1 SARIMA

The SARIMA model was fitted using `auto.arima()` to automatically select parameters, capturing seasonal patterns which automatically identifies the optimal parameters for the SARIMA model based on the training data. After fitting the model to the training data, the `forecast()` function generates predictions for future time periods, with the forecast set to match the length of the test dataset to facilitate comparison between predicted and actual values allowing us to evaluate how accurately the model could predict unseen values as shown in figures 20 -23.

```
# Apply SARIMA using auto.arima() function
sarima_model <- auto.arima(train, seasonal = TRUE)

# Forecast using the SARIMA model
sarima_forecast <- forecast(sarima_model, h = length(test))

# Plot the forecast
plot(sarima_forecast)
```

Figure 20 SARIMA fitting and forecasting

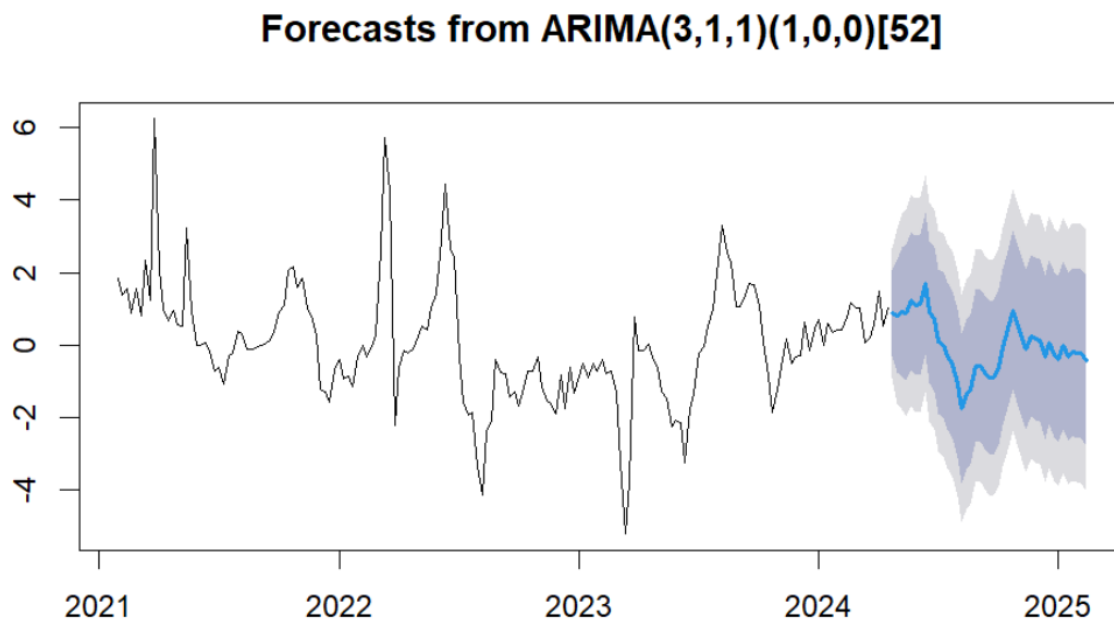


Figure 21. ARIMA forecast plot

```
{r}
# Create comparison data frame
sarima_df <- data.frame(
  week = 1:length(test),
  Actual = as.numeric(test),
  Predicted = as.numeric(sarima_forecast$mean)
)

ggplot(sarima_df, aes(x = week)) +
  geom_line(aes(y = Actual, colour = "Actual")) +
  geom_line(aes(y = Predicted, colour = "Predicted")) +
  labs(
    title = "SARIMA: Actual vs Predicted Fuel Prices (Test Set)",
    x = "Test Week",
    y = "Differenced Fuel Price"
  ) +
  scale_colour_manual(values = c("Actual" = "black", "Predicted" = "red")) +
  theme_minimal()
```

Figure 22. SARIMA actual vs predicted code

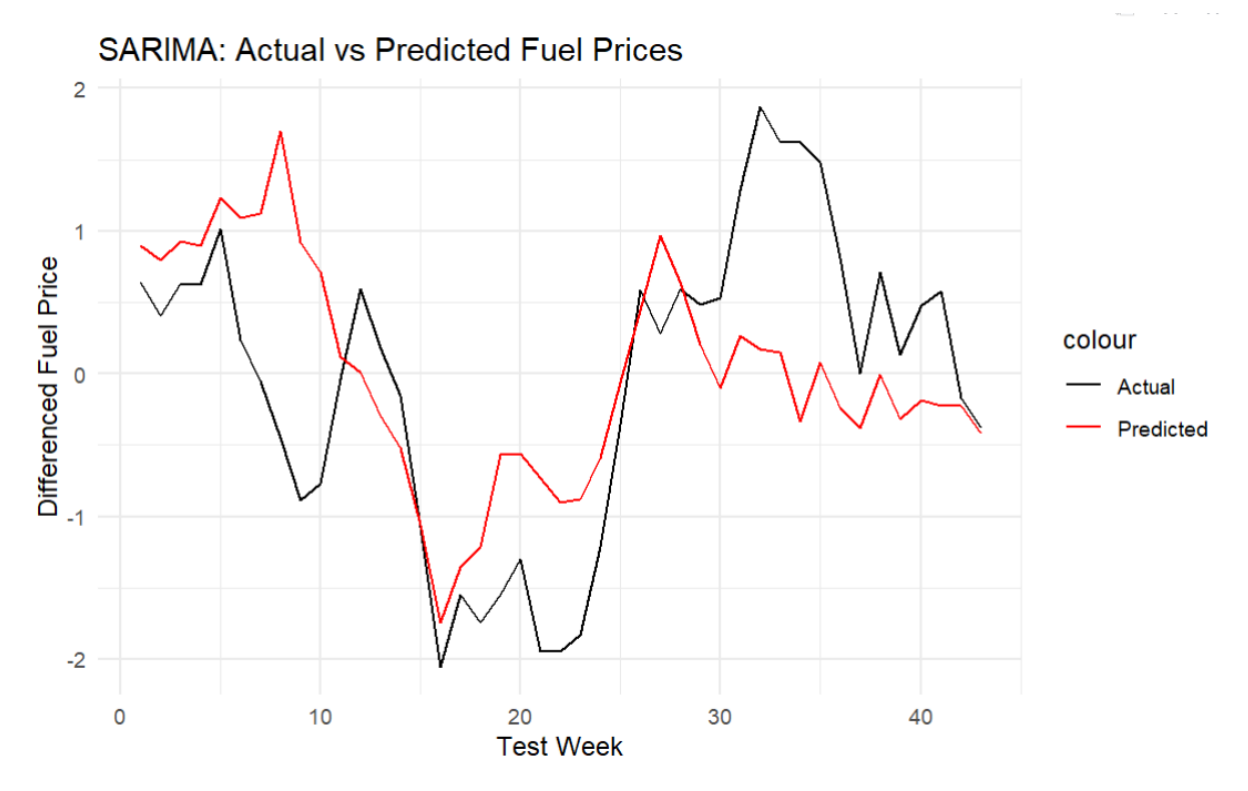


Figure 23. SARIMA actual vs predicted plot

4.2.2 Holt-Winters

Using the `HoltWinters` function, the model is fitted to historical training data as shown in figure 24 below, forecasts for future time periods are then generated with the prediction horizon set to the length of the test dataset. This enables a direct evaluation of forecast accuracy by comparing predicted values against actual observations.

```
{r}  
hw_model <- Holtwinters(train)  
hw_forecast <- forecast(hw_model, h = length(test))
```

Figure 24. Fitting Holt - Winters

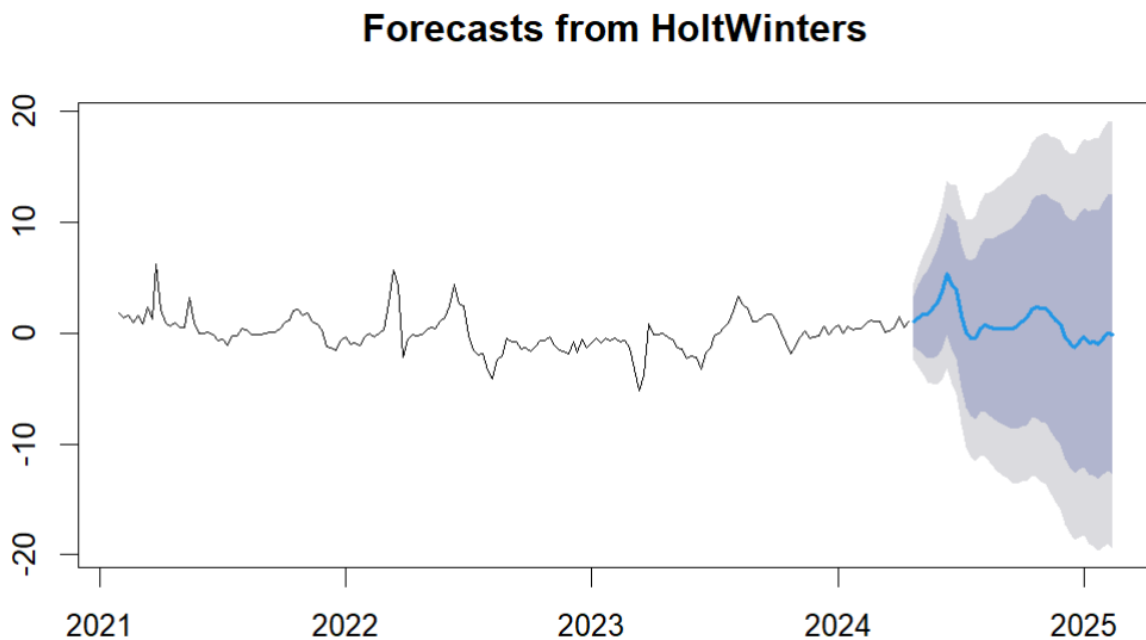


Figure 25. Holt - Winters forecast

4.2.3 STL

STL (Seasonal-Trend decomposition using Loess) method is used to model and forecast time series data. The `stl` function is applied to the training dataset, which automatically decomposes the series into seasonal, trend, and residual components using LOESS (Locally Estimated Scatterplot Smoothing). The method is particularly suited to time series with pronounced seasonality and we align the forecast with the length of the test set like previous model to enable direct performance evaluation.

```
{r}
stl_model <- stl(train, h = length(test))
stl_forecast <- stl_model
plot(stl_forecast)
```

Figure 26. Fitting STL

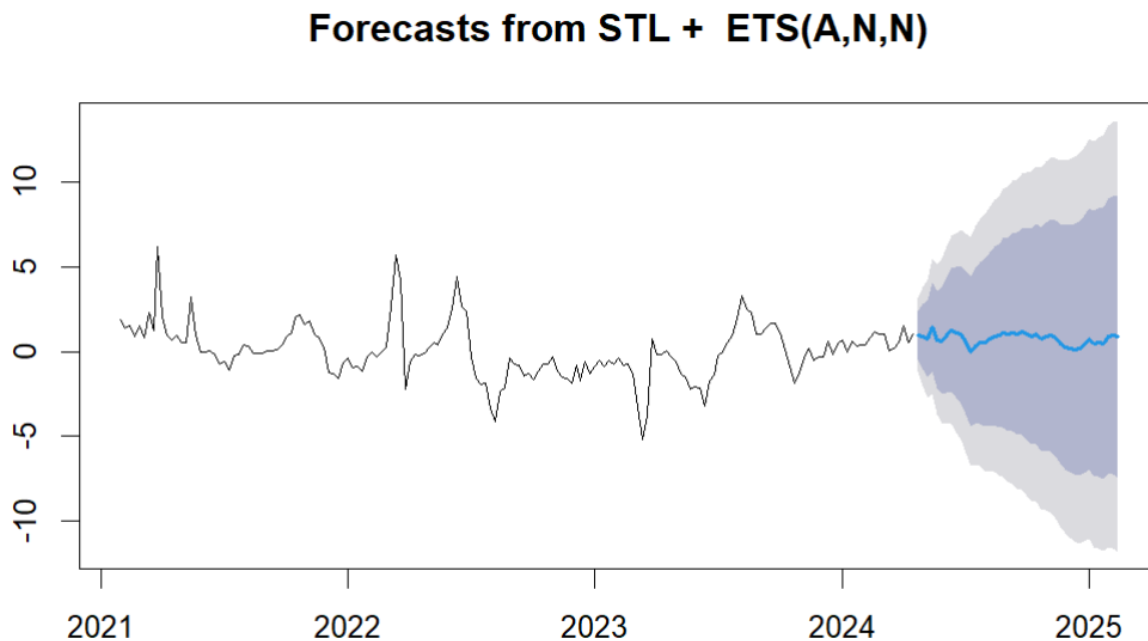


Figure 27. STL plot

4.3 Multivariate Time Series Analysis

Exogenous variables can improve predictive accuracy, although univariate models are useful for identifying patterns within a single variable, they often fall short when external factors strongly influence the series. In the case of fuel pricing, it is established that external drivers such as global oil prices or consumption behaviour can significantly impact local retail prices (Kilian and Zhou, 2020). To address the influence of external factors on UK fuel prices and enhance forecasting accuracy, multivariate time series models were employed, incorporating Brent crude oil prices as an exogenous variable, the exogenous variable was cleaned, imputing missing values via linear interpolation to ensure a complete series. The data was merged with the fuel dataset by week ending dates using left join to form a unified dataset for modelling. This structure allowed us to evaluate both direct and lagged effects of exogenous factors on weekly UK fuel prices.

```

{r}
# Load the external data (oil prices)
external_data <- read_csv("DCOILBRETEU.csv")

external_data <- external_data |>
  mutate(observation_date = as.Date(observation_date, format = "%Y-%m-%d"))

sum(is.na(external_data$DCOILBRETEU))

# Handle missing values
external_data <- external_data |>
  mutate(DCOILBRETEU = na.approx(DCOILBRETEU, na.rm = FALSE))

sum(is.na(external_data$DCOILBRETEU))

# Standardise week alignment: set both datasets to the same weekday
fuel <- fuel |>
  mutate(week_ending = floor_date(week_ending, unit = "week", week_start = 5))

external_weekly <- external_data |>
  mutate(week_ending = floor_date(observation_date, unit = "week", week_start = 5))
|>
  group_by(week_ending) |>
  summarise(Oil_Price = mean(DCOILBRETEU, na.rm = TRUE))

# Merge
merged_data <- left_join(fuel, external_weekly, by = "week_ending")

head(merged_data)

```

Figure 28 External Data

While all models discussed in this section are used for multivariate forecasting, their multivariate capability depends on how they are applied. SARIMAX, Dynamic Linear Models (DLM), and Vector Autoregression (VAR) are explicitly designed for multivariate time series analysis, where relationships between multiple variables evolve over time. On the other hand, machine learning models such as Support Vector Regression (SVR) and Random Forests become multivariate only when trained with multiple predictors (Hyndman and Athanasopoulos, 2021; Lütkepohl, 2007).

4.3.1 SARIMAX

To improve the accuracy of our fuel price forecasts, we extended the SARIMA model by incorporating external variable in the SARIMAX model. Using the `auto.arima()` function in R with

the xreg argument, we trained the SARIMAX model on an 80% training subset of the data. The model automatically selected optimal values for the autoregressive, differencing and moving average components, along with their seasonal counterparts. The key advantage of SARIMAX is that it maintains the time-series structure of SARIMA while incorporating meaningful exogenous variables, enabling it to model both internal dynamics and external drivers simultaneously (Hyndman and Athanasopoulos, 2021).

Forecasts are generated using the forecast() function, which leverages the trained SARIMAX model and future oil price values from the test dataset. The SARIMAX forecast output provides probability based predictions, including single value predictions (mean values) and confidence ranges. The code then extracts these mean values using the \$mean property, which reflects the projected average fuel prices for the forecast period.

```
# SARIMAX
sarimax_model <- auto.arima(train_ts[, "avg_fuel_price"], xreg = train_ts[,
"oil_price"])
sarimax_forecast <- forecast(sarimax_model, xreg = test_ts[, "oil_price"])
sarimax_pred <- sarimax_forecast$mean
```

Figure 29 Fitting SARIMAX

4.3.2 Dynamic Linear Model (DLM)

In the DLM process, the function (build_dlm) defines how the model handles randomness in observations (dV) and system evolution (dW), The dlmMLE function finds the best-fitting parameters for the training data, starting with initial guesses (both set to 1) for these variance components. The optimised model generates predictions for the same number of periods as the

test dataset. The final predictions are extracted as the model best estimate of future fuel prices from the forecast outputs. This approach provides a flexible framework for modelling time series where the underlying trend might change gradually over time, while automatically determining the optimal balance between tracking recent changes and filtering out random noise (Koopman and Durbin, 2012).

```
build_dlm <- function(par) {
  dlmModPoly(order = 1, dV = exp(par[1]), dW = exp(par[2]))
}

dlm_fit <- dlmMLE(train_ts[, "avg_fuel_price"], parm = c(1, 1), build = build_dlm)
model_dlm <- build_dlm(dlm_fit$par)
dlm_forecast <- dlmForecast(model_dlm, nAhead = nrow(test_ts))
dlm_pred <- sapply(dlm_forecast$a, function(x) x[1])
```

Figure 30 Fitting DLM

4.3.3 VAR

To implement a Vector Autoregression (VAR) model to forecast average fuel prices while accounting for relationships between multiple time series variables. First, the VARselect function identifies the optimal number of historical time lags (lags) to include in the model, evaluating up to 10 previous time periods and selecting the best configuration based on statistical criteria like the Akaike Information Criterion (AIC) which is identified as 3 as shown in figure 31. Forecasts are generated for the same number of periods as the test dataset. Finally, the code extracts the specific forecast values for average fuel prices from the results, focusing on the point estimates (fcst) to compare against actual test observations.

\$selection			
AIC(n)	HQ(n)	SC(n)	FPE(n)
3	3	1	3

Figure 31 VARSelect result

```
VARselect(train_ts, lag.max = 8, type = "const") # choose best lag
model_var <- VAR(train_ts, p = 3)
var_forecast <- predict(model_var, n.ahead = nrow(test_ts))
var_pred <- var_forecast$fcst$avg_fuel_price[, "fcst"]
```

Figure 32 Fitting VAR

4.3.4 SVR

To prepare data for a Support Vector Regression (SVR) model, temporal dependencies from historical fuel prices and oil prices is incorporated. First, the differenced time series data is converted to a data frame to enable feature engineering. Three lagged variables are created:

- lag1: The average fuel price from the previous time period
- lag2: The average fuel price from two periods prior
- oil_lag1: The oil price from the previous period

The SVR model is trained on the lagged variables, this structure allows the model to learn how fuel prices evolve based on their own recent history and delayed oil price effects. Predictions are generated for the test period by feeding the lagged test data into the trained model. The approach combines time series dynamics with machine learning flexibility, enabling non-linear pattern recognition.

```

# Convert merged_ts_diff to a data frame
merged_ts_diff_df <- data.frame(merged_ts_diff)

# Apply lag and create new variables (lag1, lag2, oil_lag1)
svr_df <- merged_ts_diff_df %>%
  mutate(
    lag1 = lag(avg_fuel_price, 1),      # Lag of avg_fuel_price by 1
    lag2 = lag(avg_fuel_price, 2),      # Lag of avg_fuel_price by 2
    oil_lag1 = lag(oil_price, 1)        # Lag of Oil_Price by 1
  ) %>%
  drop_na() # Drop NA values generated by lagging

head(svr_df)

train_svr <- svr_df[1:floor(0.8 * nrow(svr_df)), ]
test_svr <- svr_df[(floor(0.8 * nrow(svr_df)) + 1):nrow(svr_df), ]

```

Figure 33. Formatting the data for SVR and RF

```

# Train the SVR model with lagged fuel price and oil price as exogenous variables
svr_model <- svm(avg_fuel_price ~ lag1 + lag2 + oil_lag1, data = train_svr)
svr_pred <- predict(svr_model, newdata = test_svr)

```

Figure 34. Fitting SVR

4.3.5 Random Forest

The code in figure 35 trains a Random Forest regression model to predict fuel prices. This ensemble learning method builds multiple decision trees and combines their outputs to generate more stable and accurate forecasts. We used lagged values of fuel prices and Brent crude oil prices as input features, allowing the model to learn from both historical price patterns and external market signals. The method aggregates predictions from multiple decision trees trained on 80% of the data, generating test forecasts that balance non-linear pattern recognition with reduced overfitting risks.

```
{r}
# Random Forest
rf_model <- randomForest(avg_fuel_price ~ lag1 + lag2 + oil_lag1, data = train_svr)
rf_pred <- predict(rf_model, newdata = test_svr)
```

Figure 35 Fitting RF

4.4 Model Evaluation

To determine the most accurate forecasting approach for UK fuel prices, seven models were implemented and evaluated: SARIMA, STLF, Holt-Winters (traditional univariate approaches) and VAR, DLM, SARIMAX, SVR, and Random Forest (multivariate approaches incorporating exogenous variables). All models were trained using an 80/20 train and test split to ensure robust validation.

The performance of each model was assessed using error metrics Root Mean Square Error (RMSE) which measures the square root of the average squared differences between predicted and actual values and Mean Absolute Error (MAE) which represents the average absolute difference between predicted and actual values.

SVR performs the best with the lowest RMSE - 0.5089 and MAE - 0.4270, followed closely by Random Forest RMSE - 0.6172 and MAE - 0.5061, Holt-Winters performs the worst with RMSE - 2.1801 and MAE - 1.7899, possibly due to complex patterns, SARIMA and SARIMAX, while better than Holt-Winters, still lag behind machine learning models like SVR and Random Forest.

Model <chr>	RMSE <dbl>	MAE <dbl>
SARIMA	0.9033	0.7178
Holt-Winters	2.1801	1.7899
STLF	1.5106	1.1940
VAR	1.0497	0.8447
DLM	1.0525	0.8568
SARIMAX	1.5302	1.2283
SVR	0.5089	0.4270
Random Forest	0.6172	0.5061

Figure 36 Model evaluation

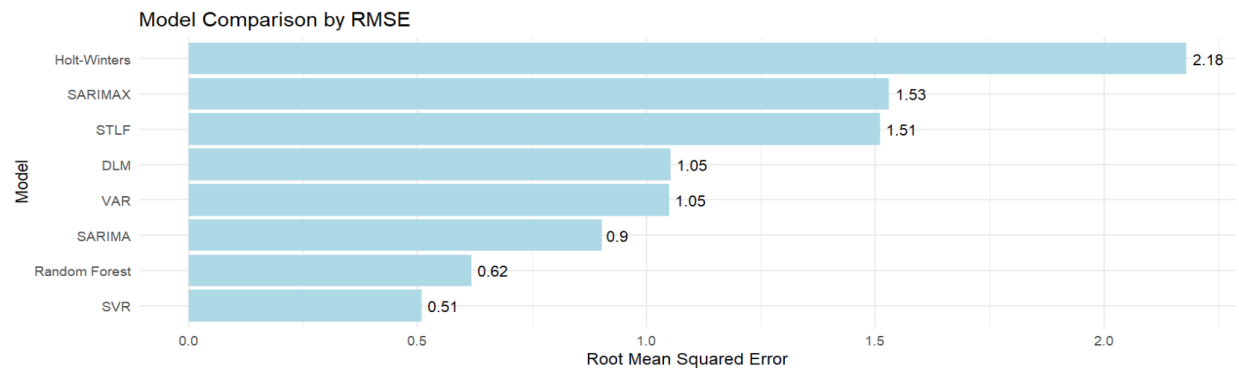


Figure 37 Model comparison bar plot

Plotting the best performing model predicted and actual values in the test set showed how closely SVR predicted values are to the actual value.

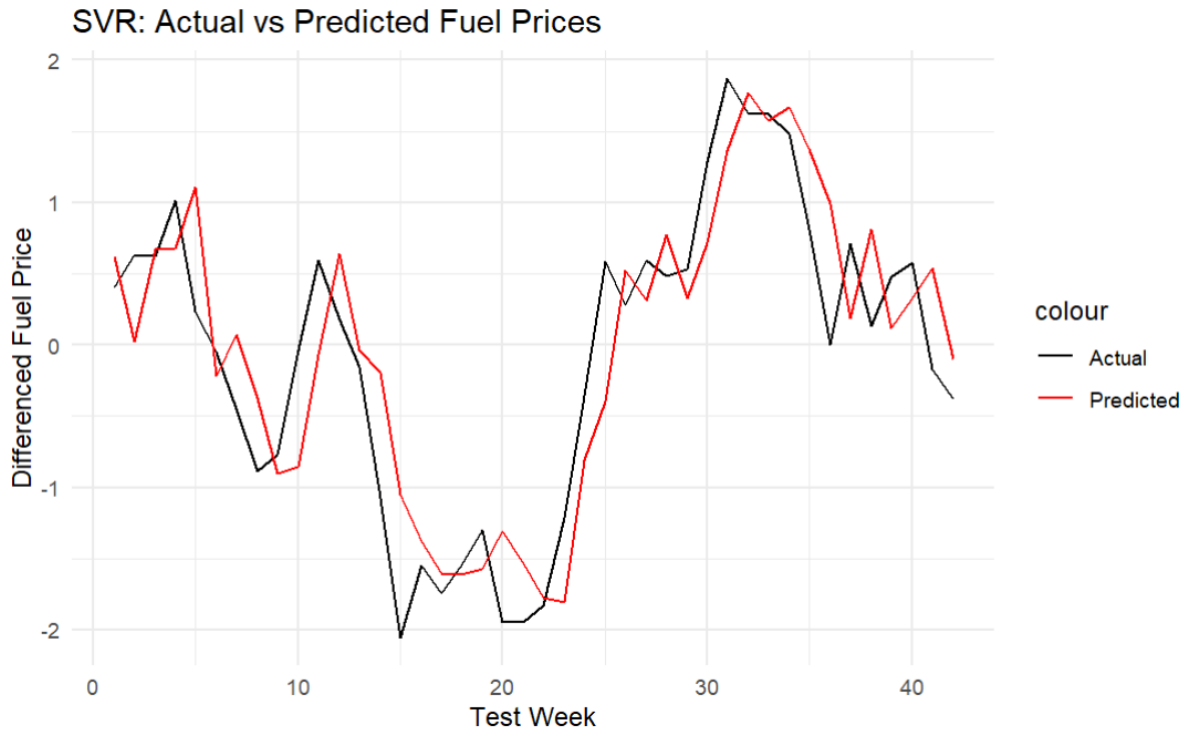


Figure 38 SVR: Actual vs Predicted

4.5 Forecast

Following the model evaluation, the Support Vector Regression (SVR) model was selected to generate the final 24-week forecast due to its superior performance in both RMSE and MAE metrics. To maximise predictive power, the model was retrained on the entire dataset, including the former test set, ensuring it learned from the full range of historical trends and patterns. As shown in Figures 38 and 39, the forecasts suggested a moderate decline in fuel prices.

	Week_Ending	Forecasted_Fuel_Price
1	2025-02-21	98.23469
2	2025-02-28	98.06561
3	2025-03-07	97.90260
4	2025-03-14	97.74507
5	2025-03-21	97.59247
6	2025-03-28	97.44425
7	2025-04-04	97.29995
8	2025-04-11	97.15910
9	2025-04-18	97.02128
10	2025-04-25	96.88609
11	2025-05-02	96.75315
12	2025-05-09	96.62208
13	2025-05-16	96.49254
14	2025-05-23	96.36419
15	2025-05-30	96.23670
16	2025-06-06	96.10973
17	2025-06-13	95.98297
18	2025-06-20	95.85609
19	2025-06-27	95.72875
20	2025-07-04	95.60063
21	2025-07-11	95.47138
22	2025-07-18	95.34063
23	2025-07-25	95.20801
24	2025-08-01	95.07312

Figure 39. Forecasted SVR values

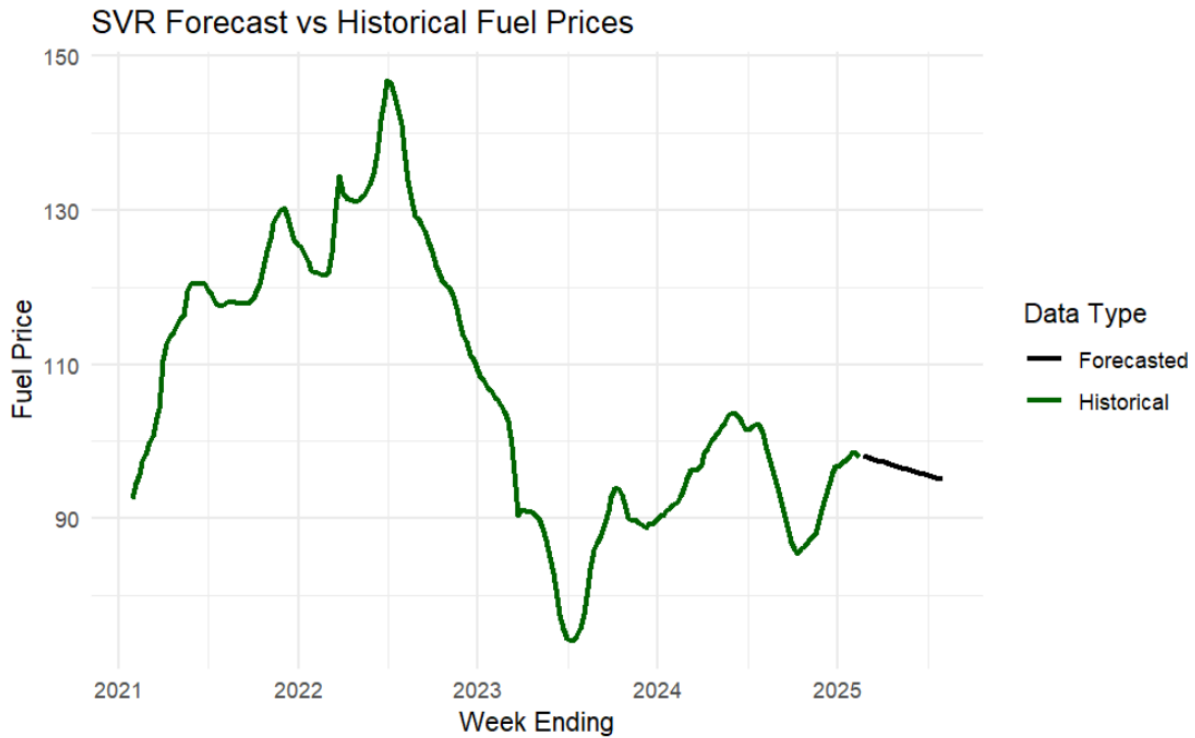


Figure 40. SVR historical and forecasted plot

4.6 Summarisation of Business Question I

Are there trends and seasonal patterns in UK average fuel prices?

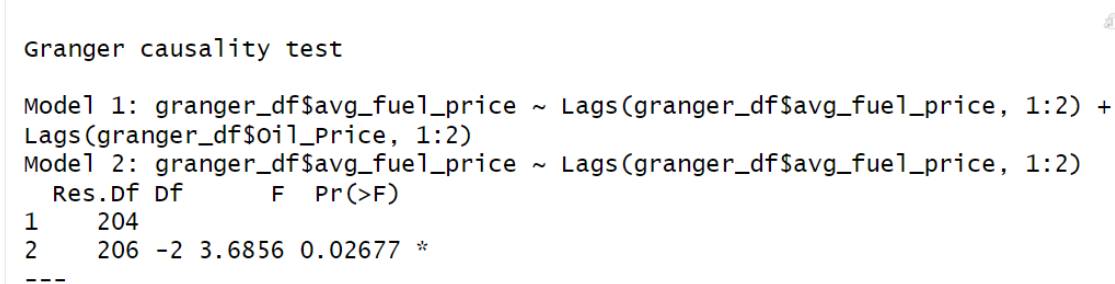
STL decomposition in Figure 13 revealed a strong seasonal pattern in UK fuel prices, recurring every 52 weeks, but showed no significant long-term trend. This confirms that fuel prices are influenced by annual cycles rather than a sustained increase or decrease over time. The presence of seasonality justifies the use of models like SARIMA and STLF for accurate forecasting.

4.7 Summarisation of Business Question II

Do crude oil prices influence average fuel prices in the UK?

To assess the relationship between Brent crude oil prices and UK average fuel prices, we performed a Granger causality test shown in Figure 41. The results showed a statistically significant relationship ($p = 0.02677$), indicating that changes in crude oil prices Granger-cause fuel price movements in the UK., confirming their predictive utility and validating their use as an exogenous variable.

```
# Granger causality test
granger_test_oil_to_fuel <- grangertest(granger_df$avg_fuel_price ~
granger_df$oil_Price, order = 2)
print(granger_test_oil_to_fuel)
```



```
Granger causality test

Model 1: granger_df$avg_fuel_price ~ Lags(granger_df$avg_fuel_price, 1:2) +
Lags(granger_df$oil_Price, 1:2)
Model 2: granger_df$avg_fuel_price ~ Lags(granger_df$avg_fuel_price, 1:2)
  Res.Df Df      F Pr(>F)
1     204
2     206 -2  3.6856 0.02677 *
```

Figure 41 Granger causality

4.8 Summarisation of Business Question III

Does average road fuel sales volume influence the accuracy of multivariate time-series forecasts?

SARIMAX models were fitted with and without road fuel sales to evaluate the impact of the variable. Including road fuel sales significantly improved the model forecasting accuracy, reducing RMSE from 1.53 to 1.07. Road fuel sales improve multivariate model performance by capturing demand dynamics not explained by crude oil prices alone.

```

# Fit SARIMAX model with both oil prices and sales volume as exogenous variables
sarimax_with_sales <- auto.arima(train_ts[, "avg_fuel_price"], xreg = train_ts[, c
("avg_rd_sales", "Oil_Price")], seasonal = TRUE)
sarimax_with_sales_forecast <- forecast(sarimax_with_sales, xreg = test_ts[, c
("avg_rd_sales", "Oil_Price")])

# Fit SARIMAX model with only oil prices as the exogenous variable
sarimax_no_sales <- auto.arima(train_ts[, "avg_fuel_price"], xreg = train_ts[,
"Oil_Price", drop = FALSE], seasonal = TRUE)
sarimax_no_sales_forecast <- forecast(sarimax_no_sales, xreg = test_ts[, "Oil_Price"
, drop = FALSE])

actual_f <- test_ts[, "avg_fuel_price"]
rmse_sarimax_with_sales <- rmse(actual_f, sarimax_with_sales_forecast$mean[1:length
(actual_f)])
rmse_sarimax_no_sales <- rmse(actual_f, sarimax_no_sales_forecast$mean[1:length
(actual_f)])

compare_sales_df <- data.frame(
  Model = c("With Sales Volume", "Without Sales Volume"),
  RMSE = c(rmse_sarimax_with_sales, rmse_sarimax_no_sales)
)
print(compare_sales_df)

```

Figure 42 Road sales influence

Model <chr>	RMSE <dbl>
With Sales Volume	1.067895
Without Sales Volume	1.530154

Figure 43 Road sales influence result

4.9 Summarisation of Business Question IV

Which forecasting model provides the best accuracy for UK fuel prices?

Among the models evaluated, Support Vector Regression (SVR) achieved the highest accuracy, with the lowest RMSE (0.5089) and MAE (0.4270) shown in Figure 36 and 37, followed closely by Random Forest. These machine learning models outperformed traditional statistical approaches like SARIMA and Holt-Winters

4.10 Summarisation Business of Question V

What are the forecasted average fuel prices for 24 weeks into the future using historical patterns?

The best model (SVR) was refit on the full dataset and used for a 24 - week recursive forecast .The SVR model forecasts a moderate fall in fuel prices over the next 24 weeks as shown in Figure 39, providing actionable insights for pricing, logistics and budgeting.

5. Conclusion and Recommendations

This report addressed the challenge of forecasting UK weekly average fuel prices using both traditional time-series models and machine learning approaches. Through extensive data preprocessing and exploratory analysis, a clear seasonal structure was identified in the data, repeating every 52 weeks, with no sustained long-term trend. Incorporating external variables such as Brent crude oil prices and road fuel sales enhanced model performance, with Granger causality confirming that crude oil prices significantly influence domestic fuel prices (Kilian and Zhou, 2020). Among all models evaluated, Support Vector Regression (SVR) had the most accurate forecasts, outperforming both classical models like SARIMA and modern machine learning models such as Random Forest. The final 24-week forecast produced by SVR suggested a moderate decline in fuel prices, providing valuable guidance for fuel retailers, logistics planners, and policymakers. These findings highlight the importance of integrating external economic indicators into forecasting systems.

Based on the results, it is recommended that SVR-based models be adopted for operational use due to their flexibility in capturing complex, non-linear relationships. Forecast accuracy can be further improved by integrating a broader range of exogenous variables, such as exchange rates, or fuel tax changes (Hyndman and Athanasopoulos, 2021). Future research should explore hybrid approaches that combine SARIMA interpretability with SVR adaptability (Alharbi and Csala, 2022). Retailers can use these forecasts to optimise pricing and stock management, while logistics providers benefit by aligning transport planning with anticipated fuel costs. Policymakers, in turn, can use insights from these models to design fuel stabilisation schemes and inflation-mitigation

strategies. Overall, the deployment of accurate and responsive forecasting tools is essential for enhancing economic resilience in the face of fuel price volatility.

6. Part B: Solution to Optimisation problem

6.1 Problem Statement

Supply chain logistics often represent a significant portion of operational costs for organisations with complex distribution networks. This project addresses the challenge of minimising transportation costs while meeting all delivery requirements across multiple routes and transportation modes.

The business currently utilises a combination of transportation modes (Road, Air, Rail, and Sea) to deliver products across three key routes (Route A, Route B, and Route C). The current transportation plan has evolved organically without systematic optimisation, resulting in potentially inefficient allocation of resources and elevated costs.

6.2 Optimisation Problem

The key challenge is to determine the optimal allocation of shipments across available transportation modes for each route to minimise total shipping costs while ensuring:

- a. All order quantities are delivered in full.
- b. Transportation mode capacity constraints are respected.

The transportation cost minimisation problem can be formulated as a linear programming model

6.2.1 Objective Function

$$\text{Minimise } Z = \sum_a \sum_\beta C_{a\beta} \times X_{a\beta}$$

Where:

$C_{a\beta}$ represents the shipping cost per unit from Route a using Transportation Mode β

$X_{a\beta}$ represents the quantity shipped from Route a using Transportation Mode β

6.2.2 Constraints

- a. Route Demand Constraints: $\sum_{\beta} X_{a\beta} = D_a$ for all routes a

Where D_a is the total demand for route a

- b. Transportation Mode Capacity Constraints: $\sum_a X_{a\beta} \leq M_{\beta}$ for all transportation modes β

Where M_{β} is the capacity of transportation mode β

- c. Non-negativity Constraints: $X_{a\beta} \geq 0$ for all routes a and transportation modes β
- d. Integer Constraints: $X_{a\beta}$ must be integers for all routes a and transportation modes β

6.2.3 Decision Variables

$X_{a\beta}$: Quantity shipped from route a using transportation mode β

Where $a \in \{\text{Route A, Route B, Route C}\}$

And $\beta \in \{\text{Road, Air, Rail, Sea}\}$

6.2.4 Parameters

$C_{a\beta}$: Average shipping cost per unit for route a using transportation mode β

D_a : Total demand for route a

M_{β} : Maximum capacity for transportation mode β

6.3 Implementation

The optimisation problem was implemented using R with the lpSolve package for linear programming.

6.3.1 Data Preparation

To begin implementation the data is read into R and the relevant columns are extracted from the supply chain dataset

```
data <- read.csv("supply_chain_data.csv", stringsAsFactors = FALSE)

# Select relevant columns
transport_data <- data %>%
  select(
    SKU,
    Product_type = Product.type,
    Order_quantities = Order.quantities,
    Routes,
    Transportation_modes = Transportation.modes,
    Shipping_costs = Shipping.costs
  )
```

Figure 44 Reading and reprocessing data for optimisation

```
# Get unique routes and transportation modes
unique_routes <- unique(transport_data$Routes)
unique_modes <- unique(transport_data$Transportation_modes)
```

Figure 45 Extracting unique routes and transportation modes

```
# Calculate average shipping costs by route and transportation mode
cost_summary <- transport_data %>%
  group_by(Routes, Transportation_modes) %>%
  summarize(
    Avg_Cost = mean(Shipping_costs),
    Total_Order_Qty = sum(Order_quantities),
    .groups = "drop"
  )
```

Figure 46 Average shipping costs by route and mode

	Routes	Transportation_modes	Order_Quantity	Current_Cost
1	Route A	Air	486	2850.845
2	Route A	Rail	573	2878.870
3	Route A	Road	432	2294.773
4	Route A	Sea	353	1485.480
5	Route B	Air	485	2637.251
6	Route B	Rail	608	3973.586
7	Route B	Road	720	4023.117
8	Route B	Sea	253	1210.398
9	Route C	Air	370	1822.150
10	Route C	Rail	161	1076.016
11	Route C	Road	234	1262.983
12	Route C	Sea	247	1822.478

Figure 47 Current costs by Routes and Transportation Mode

```
# Calculate current total cost
current_cost_data <- transport_data %>%
  mutate(Total_Cost = Order_quantities * Shipping_costs) %>%
  group_by(Routes, Transportation_modes) %>%
  summarize(
    Order_Quantity = sum(Order_quantities),
    Current_Cost = sum(Total_Cost),
    .groups = "drop"
  )

# Calculate total current cost before optimisation
cat("Total Cost Before Optimization:", sum(current_cost_data$Current_Cost))
```

Figure 48 Total current cost code snippet

```
> # Calculate total current cost before optimization
> cat("Total Cost Before Optimization:", sum(current_cost_data$Current_Cost))
Total Cost Before Optimization: 27337.95
```

Figure 49 Total current cost

6.3.2 Model Setup

```
# Create a matrix to store shipping costs by route and mode
cost_matrix <- matrix(0, nrow = length(unique_routes), ncol = length(unique_modes))
rownames(cost_matrix) <- unique_routes
colnames(cost_matrix) <- unique_modes

# Fill the cost matrix with average shipping costs for each route-mode combination
for(route in unique_routes) {
  for(mode in unique_modes) {
    subset_data <- cost_summary %>%
      filter(Routes == route, Transportation_modes == mode)

    if(nrow(subset_data) > 0) {
      cost_matrix[route, mode] <- subset_data$Avg_Cost
    } else {
      # If no data for this combination, set a high cost
      cost_matrix[route, mode] <- 1000
    }
  }
}
```

Figure 50 Creating the cost matrix

```
# Calculate total order quantities for each route
route_demand <- transport_data %>%
  group_by(Routes) %>%
  summarize(total_demand = sum(Order_quantities))

# Calculate total demand
total_demand <- sum(route_demand$total_demand)
```

Figure 51 Quantity to ship per route-mode combination

The total quantity to ship is 4922

```
# Set mode capacities
mode_capacity <- c(
  "Air" = total_demand * 0.4,
  "Rail" = total_demand * 0.35,
  "Road" = total_demand * 0.4,
  "Sea" = total_demand * 0.25
)
```

Figure 52 Defining capacity constraints for each transportation mode

```
> mode_capacity
      Air   Rail   Road   Sea
1968.8 1722.7 1968.8 1230.5
```

Figure 53 Mode Capacity

```
# Route constraints (= demand)
route_constraints <- matrix(0, nrow = num_routes, ncol = num_vars)
for(i in 1:num_routes) {
  for(j in 1:num_modes) {
    route_constraints[i, (i-1)*num_modes + j] <- 1
  }
}

# Mode constraints (<= capacity)
mode_constraints <- matrix(0, nrow = num_modes, ncol = num_vars)
for(i in 1:num_routes) {
  for(j in 1:num_modes) {
    mode_constraints[j, (i-1)*num_modes + j] <- 1
  }
}

# Combine constraints
constraint_matrix <- rbind(route_constraints, mode_constraints)
```

Figure 54 Setting route demand constraints to ensure all orders are fulfilled

6.4 Result and Analysis

6.4.1 Solution Matrix

```
# Solve the linear programming problem
lp_solution <- lp(
  direction = "min",
  objective.in = cost_vector,
  const.mat = constraint_matrix,
  const.dir = constraint_dir,
  const.rhs = rhs_values,
  all.int = TRUE |
)
```

Figure 55 Solving the linear programming problem

```
# Extract solution
solution_matrix <- matrix(lp_solution$solution, nrow = num_routes, ncol = num_modes)
rownames(solution_matrix) <- unique_routes
colnames(solution_matrix) <- unique_modes

# Calculate total cost
total_cost <- sum(solution_matrix * cost_matrix)
```

Figure 56 Extracting the optimal solution

The optimisation model has successfully identified an optimal transportation plan that results in significant cost savings shown in Figure 57 below.

```
> solution_matrix
      Road  Air Rail  Sea
Route B    0  218  98   0
Route C 1626 1968   0   0
Route A    0    0   0 1012
```

Figure 57 Solution Matrix

6.4.2 Cost Comparison

The optimisation model reduced the total shipping cost from the original value to a minimum of 24,743.29, resulting in a cost saving of 2,594.66, which represents an overall efficiency gain of 9.49% compared to the initial scenario.

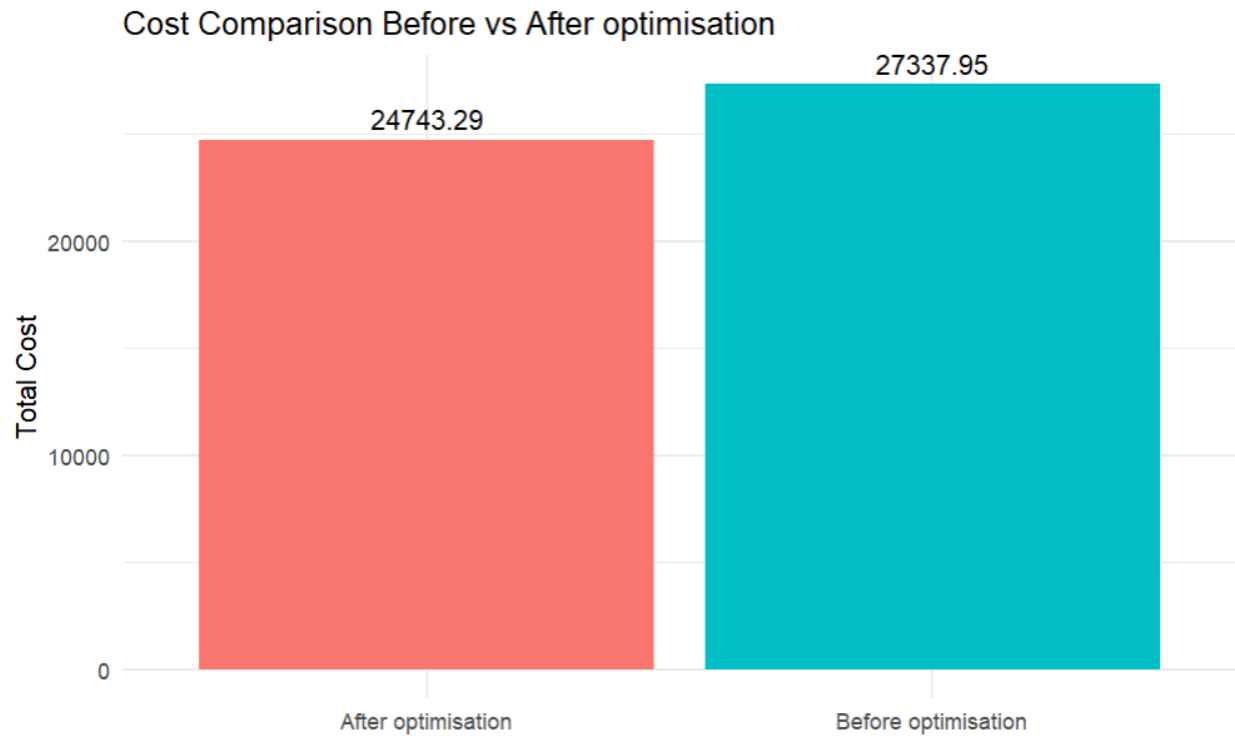


Figure 58 Cost Comparison

6.4.3 Mode Utilisation Analysis

The optimisation has resulted in a more efficient allocation of transportation modes

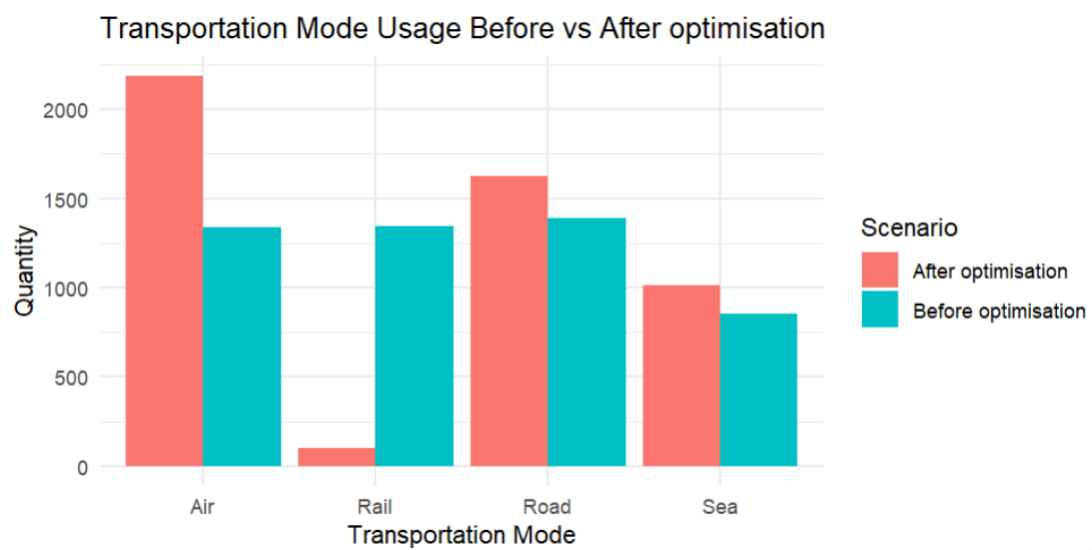


Figure 59 Mode Utilisation Analysis

Before Optimisation the transportation modes were, Air: 1341 units, Rail: 1342 units, Road: 1386 units, Sea: 853 units.

After Optimisation, Air: 2,186 units, Rail: 98 units, Road: 1626 units, Sea: 1012 units.

6.4.4 Route Utilisation Analysis

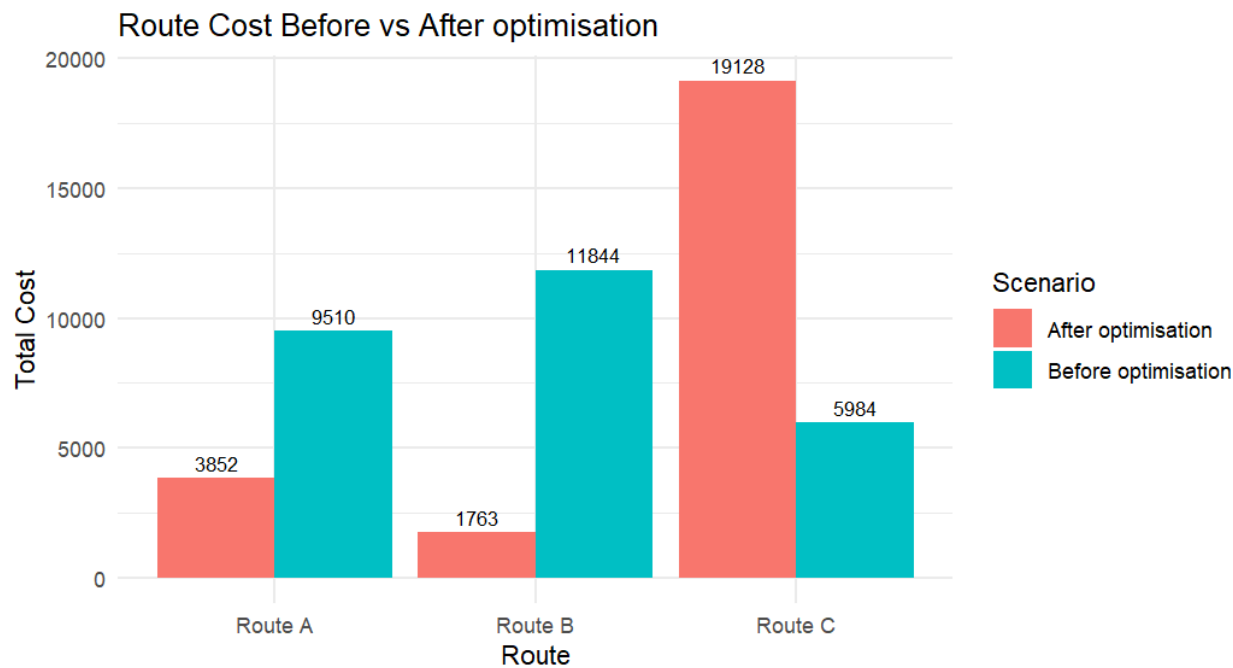


Figure 60 Route Utilisation Analysis

6.4.5 Key Insights

The optimisation results reveal clear strategic shifts in transportation allocation. Route A is now served exclusively by Sea transport, capitalising on its cost-effectiveness and suitability for bulk shipment. Route B adopts a balanced mix of Air and Rail, optimising for a trade-off between delivery speed and cost efficiency. Meanwhile, Route C uses a combination of Road and Air transport, likely reflecting operational constraints or route-specific service requirements.

6.4.6 Constraint Satisfaction

The solution satisfies all specified constraints:

- a. All orders (4,922 units) are delivered in full
- b. None of the transportation mode capacities are exceeded
- c. The solution provides practical, implementable quantities

6.5 Recommendations

Based on the optimisation results, several strategic recommendations are proposed to maximise cost savings and operational efficiency. First, transport mode allocation should be adjusted as follows: transition all shipments on Route A to Sea transport (1,012 units), use a mix of Air (218 units) and Rail (98 units) for Route B and implement an optimal blend of Road (1,626 units) and Air (1,968 units) for Route C. To ensure a smooth transition, a phased implementation is advised, beginning with focus on Route A conversion to Sea transport, supported by continuous monitoring to compare actual costs against forecasted savings.

6.6 Conclusion

The transportation cost optimisation project has successfully identified a more efficient allocation of shipments across transportation modes, resulting in significant cost savings while meeting all delivery requirements. The recommended implementation strategy balances immediate cost savings with practical considerations for smooth transition and long-term sustainability.

By adopting a data-driven approach to transportation planning, the organisation can continue to optimise its supply chain operations, potentially expanding the scope to include additional factors and constraints in future iterations of the model.

7. Personal Reflection

Working on this fuel price forecasting project has significantly deepened my understanding of time-series modelling because of the numerous challenges I had while working with the data, this report made me understand most of the concepts and techniques the module introduced me to particularly in applying time series, statistical and machine learning approaches to real-world economic data and working on the data to produce this report was thrilling, although I particularly enjoyed the part B more as I was having a really hard time in getting a business question and understanding it, having a breakthrough on both parts of the reports was fulfilling and I am interested in exploring more time series problems as implementing more advanced techniques such as SARIMAX, VAR, and SVR challenged me to think more critically about model assumptions and the role of exogenous variables in improving predictive performance.

One of the most valuable aspects of this project was learning to handle messy, real-world data. Tasks such as dealing with missing values, outlier detection, decomposition, and stationarity testing showed me that high quality results depend not only on the model used but also on the data preparation. Moreover, evaluating multiple models side by side taught me that the most accurate solution is often not the most complex one but the one best suited to the data and business context.

References

- Alharbi, F.R. and Csala, D. (2022) A Seasonal Autoregressive Integrated Moving Average with Exogenous Factors (SARIMAX) Forecasting Model-Based Time Series Approach. *Inventions (Basel)*, [Online] 7 (4), pp. 94 Available from: <https://www.proquest.com/docview/2756723224>. [05/01/2025].
- Baumeister, C. and Kilian, L. (2016) Understanding the Decline in the Price of Oil since June 2014. *Journal of the Association of Environmental and Resource Economists*, [Online] 3 (1), pp. 131–158 Available from: <https://www.jstor.org/stable/26544408>. [05/09/2025].
- Baumeister, C. and Kilian, L. (2015) Forecasting the Real Price of Oil in a Changing World: A Forecast Combination Approach. *Journal of Business & Economic Statistics*, [Online] 33 (3), pp. 338–351 Available from: <https://www.tandfonline.com/doi/abs/10.1080/07350015.2014.949342>. [05/02/2025].
- Baumeister, C. and Peersman, G. (2013) Time-Varying Effects of Oil Supply Shocks on the US Economy. *American economic journal. Macroeconomics*, [Online] 5 (4), pp. 1–28 Available from: <https://www.jstor.org/stable/43189560>. [05/01/2025].
- Box, G.E.P. and Jenkins, G.M. (1976) *Time series analysis : forecasting and control*. Rev. ed ed. [Online] Oakland, Calif: Holden-Day. Available from: <http://www.econis.eu/PPNSET?PPN=024939919>. [05/01/2025].
- Box, G.E.P., Jenkins, G.M., Reinsel, G.C. and Ljung, G.M. (2016) *Time series analysis*. Fifth edition ed. [Online] Hoboken: Wiley. Available from: http://bvbr.bib-bvb.de:8991/F?func=service&doc_library=BVB01&local_base=BVB01&doc_number=028

[194739&sequence=000002&line_number=0001&func_code=DB_RECORDS&service_type=MEDIA](#). [05/06/2025].

Breiman, L. (2001) *Random Forests*. ed. [Online] : .Available from: <https://> . [05/05/2025].

Chao, X., et al. (2020). The application of multivariate time series in fuel pricing. *Applied Energy*, 261, 114270

Chopra, S., & Meindl, P. (2021). *Supply Chain Management: Strategy, Planning, and Operation*.

Durbin, J. and Koopman, S.J. (2012) *Time series analysis by state space methods*. 2. ed. ed. [Online] Oxford [u.a.]: Oxford Univ. Press. Available from: <http://www.gbv.de/dms/zbw/689557248.pdf>. [05/02/2025].

Guo, J., Liu, Z., & Wang, X. (2019). Support Vector Machine based prediction of fuel prices. *Energy Reports*, 5, 1243-1252.

Gupta, N. and Nigam, S. (2020) Crude Oil Price Prediction using Artificial Neural Network. *Procedia computer science*, [Online] 170 , pp. 642–647 Available from: <https://dx.doi.org/10.1016/j.procs.2020.03.136>. [05/03/2025].

Hamilton, J. (2010) *Causes and consequences of the oil shock of 2007–08*. [Online] Available from: <http://econpapers.repec.org/paper/fipfedacq/2009-02.htm>. [05/06/2025].

Han, J., Pei, J. and Tong, H. (2023) *Data Mining Concepts and Techniques (4th Edition)*. Fourth edition ed. [Online] : Elsevier. Available from: <https://app.knovel.com/hotlink/toc/id:kpDMCTE007/data-mining-concepts/data-mining-concepts?kpromoter=Summon>. [05/07/2025].

- Harvey, A.C. (2003) *Forecasting, structural time series models and the Kalman filter*. Transferred to digital print. ed. [Online] Cambridge: Univ. Press. Available from: <https://> . [05/05/2025].
- Hyndman, R.J. and Athanasopoulos, G. (2021) *Forecasting : principles and practice*. 3rd ed ed. [Online] : OTexts. Available from: <https://> . [05/03/2025].
- James, G., Witten, D., Hastie, T. and Tibshirani, R. (2013) *An Introduction to Statistical Learning : with Applications in R*. 1st ed. [Online] New York: Springer New York. Available from: <https://library.biblioboard.com/viewer/73e14548-bd1f-11ea-abc2-0a28bb48d135>. [05/09/2025].
- Jha, N., Kumar Tanneru, H., Palla, S. and Hussain Mafat, I. (2024) Multivariate analysis and forecasting of the crude oil prices: Part I – Classical machine learning approaches. *Energy (Oxford)*, [Online] 296 , pp. 131185 Available from: <https://dx.doi.org/10.1016/j.energy.2024.131185>. [05/09/2025].
- Kilian, L. and Zhou, X. (2020) *Oil prices and stock market volatility*. The Energy Journal, 41(2), pp.1–36.
- Kilian, L. (2008) Exogenous oil supply shocks: how big are they and how much do they matter for the U.S. economy? *The review of economics and statistics*, [Online] 90 (2), pp. 216–240 Available from: <https://www.jstor.org/stable/40043142>. [05/04/2025].
- Koopman, S.J. and Durbin, J. (2012) *Time Series Analysis by State Space Methods: Second Edition*. ed. [Online] : Oxford University Press. Available from: <http://econpapers.repec.org/bookchap/oxpobooks/9780199641178.htm>. [05/09/2025].

Lin, C., & Li, X. (2015). Forecasting fuel prices using oil markets. *Energy Economics*, 48, 1-9.

Little, R.J.A. and Rubin, D.B. (2020) *Statistical analysis with missing data*. 3rd edition ed. [Online] Hoboken, NJ: Wiley. Available from: http://bvbr.bib-bvb.de:8991/F?func=service&doc_library=BVB01&local_base=BVB01&doc_number=025207876&sequence=000001&line_number=0001&func_code=DB_RECORDS&service_type=MEDIA. [05/01/2025].

Lütkepohl, H. (2007) *New introduction to multiple time series analysis*. Corr. second printing ed. [Online] Berlin ; Heidelberg ; New York: Springer. Available from: <https://> . [05/03/2025].

Moritz, S. and Bartz-Beielstein, T. (2017) imputeTS: Time Series Missing Value Imputation in R. *The R journal*, [Online] 9 (1), pp. 207 Available from: <https://journal.r-project.org/archive/2017/RJ-2017-009/RJ-2017-009.pdf>. [05/02/2025].

Moyo, C., Anyikwa, I. and Phiri, A. (2023) The Impact of Covid-19 on Oil Market Returns: Has Market Efficiency Being Violated? *International journal of energy economics and policy*, [Online] 13 (1), pp. 118–127 Available from: <http://www.econis.eu/PPNSET?PPN=1843703084>. [05/03/2025].

Putler, D.S. and Krider, R.E. A Process Model for Data Mining—CRISP-DM. (2012a) *Customer and Business Analytics*. ed. [Online] : Chapman and Hall/CRC, pp. 43–56. Available from: <https://www.taylorfrancis.com/books/9780429185687/chapters/10.1201/b12040-8>. [05/02/2025].

- Putler, D.S. and Krider, R.E. A Process Model for Data Mining—CRISP-DM. (2012) *Customer and Business Analytics*. ed. [Online] : Chapman and Hall/CRC, pp. 43–56. Available from: <https://www.taylorfrancis.com/books/9780429185687/chapters/10.1201/b12040-8>. [05/01/2025].
- Rojo, J., Rivero, R., Romero-Morte, J., Fernández-González, F. and Pérez-Badia, R. (2017) Modeling pollen time series using seasonal-trend decomposition procedure based on LOESS smoothing. *International journal of biometeorology*, [Online] 61 (2), pp. 335–348 Available from: <https://link.springer.com/article/10.1007/s00484-016-1215-y>. [05/08/2025].
- Schröer, C., Kruse, F. and Gómez, J.M. (2021) A Systematic Literature Review on Applying CRISP-DM Process Model. *Procedia computer science*, [Online] 181 , pp. 526–534 Available from: <https://dx.doi.org/10.1016/j.procs.2021.01.199>. [05/01/2025].
- Smola, A. and Schoelkopf, B. (2004) A tutorial on support vector regression. , [Online] , pp. Available from: <https://doi.org/10.1023/B:STCO.0000035301.49549.88>. [05/09/2025].
- Sterner, T. (2012) *Fuel Taxes and the Poor*. 1st ed. [Online] Oxford: RFF Press. Available from: <https://www.taylorfrancis.com/books/9781936331925>. [05/01/2025].
- Taylor, J. (2017) *Four Problems in Using CRISP-DM and How To Fix Them*. [Online] Available from: <https://www.kdnuggets.com/2017/01/four-problems-crisp-dm-fix.html>. [05/01/2025].
- Wickham, H., Cetinkaya-Rundel, M. and Grolemund, G. (2023) *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. 2nd edition. ed. [Online] : O'Reilly. Available from:

<https://www.vlebooks.com/vleweb/product/openreader?id=none&isbn=9781492097365>

. [05/09/2025].